

NeuroSentinel AI Guide

NeuroSentinel AI — Internal Technical Guide

Chapter 1: ML Pipeline, Dataset, Model Architecture, Training & Evaluation

Purpose: Internal reference document for building a formal conference/journal paper and maintaining institutional memory.

Source of truth: All content derived exclusively from project codebase, notebook cells, and verified production artifacts.

Last updated: September 2026 (Session 21+ — Post-April Deployment, UI, and Platform Update)

Previous version: April 21, 2026 (Sessions 1-20, V4.0.1)

Authors: Manikonda Adwith — KMEC, Hyderabad — G1108

1. Project Overview

1.1 What NeuroSentinel AI Is

NeuroSentinel AI is an **end-to-end clinical decision support system** for automated seizure detection from scalp EEG recordings. The system accepts `.edf` (European Data Format) files, preprocesses raw EEG signals, runs inference through a custom deep learning model, and produces structured clinical reports with explainability outputs and a full web interface for clinical review.

The system is live at: <https://neuro-sentinel-ai-6vfv.vercel.app>
Model Hub: <https://huggingface.co/manikondaadwith/neurosentinel-v4>
GitHub: <https://github.com/Manikondaadwith/kmec-ps-G1108>

1.2 Core Objective

Detect seizure events in continuous EEG recordings with:

- **High sensitivity** — catching real seizures (event-level: 73.3% on held-out test patients)
- **Low false positive rate** — minimizing false alarms (FP/h: 0.98)
- **Cross-dataset generalization** — zero-shot performance on Siena Scalp EEG (80% sensitivity)
- **Clinical-grade reporting** — structured outputs suitable for neurologist review including role-aware PDF reports, SCOUT AI assistant, and email notification

1.3 Key Claim Boundaries

[!IMPORTANT] NeuroSentinel AI is a **decision support tool only**. It does not diagnose, prescribe, or replace clinical judgment. All heuristic outputs (severity scores, seizure type classification, focal/generalized labeling) are rule-based estimates, not ground-truth labels. The system is **not FDA approved**.

1.4 Project Development Timeline

The project evolved across **21+ sessions** (April 2026 guide covered Sessions 1-20; this guide adds Sessions 21+ covering post-April work):

Phase	Sessions	Scope
Preprocessing Pipeline	1-3	Channel mapping, labeling, windowing
Model Training	4-6	V1/V3 (broken alpha bug), V4 production, V5 discarded
Post-Processing	7	53-config sweep → ADP_p99 winner
Universal Adapter	8-9	TTN/BNA/calibration — all failed

Cross-Dataset Testing	10-15	Siena Scalp EEG (11 files)
Explainability + Report	16	GradCAM, attention maps, clinical report
Metrics & Visualization	17	ROC/PR curves, confusion matrices
Web Deployment	18	FastAPI + Next.js + Supabase + SCOUT
Honest Metrics	19	Overfitting diagnostic, paper-ready metrics
Suspicious-State Consistency Patch	20	Diagnostic state machine, reporting reconciliation, UI fixes
Post-April Platform Expansion	21+	Logo, mobile responsive, report comparison, SCOUT sessionStorage persistence, email deliverability, PDF charts, user name personalization, Help & Support, OTP auth overhaul, Terms/Privacy pages

2. Dataset

2.1 Primary: CHB-MIT Scalp EEG Database

Property	Value
Source	PhysioNet CHB-MIT Scalp EEG Database
Patients	23 pediatric patients
Files	686 EDF recordings
Channels	22-channel bipolar montage (10-20 system)
Sampling Rate	256 Hz
Seizure Annotations	141 files with seizure annotations
Total Windows	258,196

2.2 Patient-Level Split (No Data Leakage)

The split is **patient-level** — no patient appears in multiple splits:

Split	Patients	Windows	Notes
Train	chb01-chb16	28,821	2:1 balanced (non-seizure:seizure)
Validation	chb17-chb20	30,803	Unbalanced (natural distribution)
Test	chb21-chb24	24,445	Unbalanced (natural distribution)

[!NOTE] Training data is balanced at 2:1 ratio using undersampling of non-seizure windows. Validation and test sets preserve the natural class imbalance (~16:1 non-seizure:seizure) for honest evaluation. This is **more rigorous** than random-segment splitting used in most published work.

2.3 Cross-Dataset: Siena Scalp EEG

Property	Value
Source	PhysioNet Siena Scalp EEG (via AWS S3)
Files Tested	11 EDF recordings
Montage	Referential (converted to bipolar via adapter)

Patients	14 patient IDs (PN00–PN17)
Purpose	Zero-shot cross-dataset generalization test

2.4 Standard 22-Channel Bipolar Montage

The model expects exactly 22 channels in this order (from `config.py`):

FP1-F7, F7-T7, T7-P7, P7-O1,	← Left Temporal chain
FP1-F3, F3-C3, C3-P3, P3-O1,	← Left Parasagittal chain
FP2-F4, F4-C4, C4-P4, P4-O2,	← Right Parasagittal chain
FP2-F8, F8-T8, T8-P8, P8-O2,	← Right Temporal chain
FZ-CZ, CZ-PZ,	← Midline chain
P7-T7, T7-FT9, FT9-FT10, FT10-T8	← Inferior chain

Channel-to-brain-region mapping (from `CHANNEL_TO_REGION` in `config.py`):

- Channels 0–3, 18: Left Temporal
- Channel 4: Left Frontal
- Channel 5: Left Central
- Channel 6: Left Parietal
- Channel 7: Left Occipital
- Channel 8: Right Frontal
- Channels 9–11: Right Central/Parietal/Occipital
- Channels 12–15: Right Temporal
- Channels 16–17: Midline Frontal/Parietal
- Channels 19–21: Inferior (Left/Bilateral/Right)

3. Preprocessing Pipeline

Source files: `neurosentinel-backend/app/pipeline/preprocessing.py` + notebook cells 1–28

3.1 Pipeline Overview

Raw EDF → Powerline Detection → Channel Selection (memory opt) → Load Data → Resample to 256 Hz → Bipolar Adaptation → Channel Enforcement (22ch) → Notch Filter → Bandpass Filter → IQR Normalization → [Windowing or Continuous Data Output]
--

3.2 Step 1: Powerline Frequency Detection

Function: `detect_powerline_freq()`

- Analyzes up to 4 non-overlapping 30-second segments
- Computes Welch PSD for each segment
- Calculates peak-to-floor ratio at 50 Hz vs 60 Hz candidates
- CHB-MIT (US dataset): detects 60 Hz
- Siena (European): auto-detects 50 Hz
- Default fallback: 60 Hz

3.3 Step 2: Memory-Optimized Channel Selection

Function: `_pick_needed_channels()`

Before calling `load_data()`, the pipeline identifies which channels are actually needed for the 22-channel bipolar montage and drops unnecessary channels (ECG, EMG, SpO2, etc.). This prevents allocating float64 arrays for channels that will never be used.

Non-EEG keywords filtered: `ECG, EKG, EMG, EOG, SPO2, HR, RESP, TEMP, PHOTIC, DC, BURSTS, SUPPR, IBI, BODY, PULSE, SAO2, CHIN, LEG, SNORE, FLOW, THOR, ABDO`

3.4 Step 3: Resampling

If the original sampling rate differs from the target 256 Hz by more than 0.5 Hz, the signal is resampled using MNE's polyphase resampling method.

3.5 Step 4: Bipolar Montage Adaptation

Function: `adapt_to_bipolar()`

For referential montage data (e.g., Siena), the adapter:

1. Detects montage type by counting bipolar-style channel names (contains `-` with two electrode labels)
2. If referential: normalizes electrode names (strips prefixes like `EEG`, suffixes like `-REF`, `-LE`), applies electrode aliases (`T3→T7`, `T4→T8`, `T5→P7`, `T6→P8`, `PG1→FT9`, `PG2→FT10`)
3. Computes bipolar differences: `left_electrode - right_electrode` for each of the 22 standard channels
4. If bipolar: passes data through unchanged

3.6 Step 5: Channel Enforcement

Function: `enforce_channels()`

- Maps available channel names to the standard 22-channel layout
- Creates a boolean `channel_mask` indicating which channels are present
- Zero-fills missing channels
- Rejects files with more than `MAX_MISSING_CH = 3` missing channels (31 files skipped as `skip_montage` in CHB-MIT)

3.7 Step 6: Notch Filtering

Function: `apply_notch()`

- IIR notch filter at detected powerline frequency ($Q=20$)
- Also applies at 2nd harmonic if below Nyquist
- Filter coefficients are cached via `_FILTER_CACHE`

3.8 Step 7: Bandpass Filtering

Function: `apply_bandpass()`

- 4th-order Butterworth bandpass: **0.5-40.0 Hz**
- Applied using `sosfiltfilt` (zero-phase forward-backward filtering)
- Filter coefficients cached

3.9 Step 8: Robust (IQR) Normalization

Function: `robust_normalize()`

- Computes Q25, Q50, Q75 from background (non-seizure) portions
- `robust_std = (Q75 - Q25) / 1.349` (IQR-based standard deviation estimate)
- Centers with median, scales by `robust_std`
- **Why IQR over z-score:** robust to seizure contamination in the statistics window
- If less than 10 seconds of background available, uses first 60 seconds instead

3.10 Step 9: Windowing

Function: `extract_windows_vectorised()`

Parameter	Training (Seizure)	Training (Non-Seizure)	Inference
Window size	4 seconds (1024 samples)	4 seconds (1024 samples)	4 seconds (1024 samples)
Stride	1 second (256 samples)	4 seconds (1024 samples)	1 second (256 samples)
Artifact rejection	Skipped	ON (threshold=5.0)	Skipped

Windows are extracted using `np.lib.stride_tricks.as_strided` for memory efficiency (creates a view, not a copy). The final output is copied to float32.

3.11 Two Entry Points

- 1. `preprocess_edf_to_data()` — Returns continuous `(22, n_samples)` array. Used for large files with chunked inference.
- 2. `preprocess_any_edf()` — Returns windowed `(n_windows, 22, 1024)` array. Used for smaller files or legacy paths.

[!IMPORTANT] **NEVER change the preprocessing pipeline for inference.** IQR normalization, channel ordering, and filter parameters are locked to match training conditions. Any change would constitute a data distribution shift and invalidate all declared metrics.

4. Model Architecture

Source files: `neurosentinel-backend/app/pipeline/model.py` + notebook cell 32

4.1 MultiRepEEGModel — Overview

A **3-branch fusion architecture** with **409,154 parameters**:



4.2 Branch 1: Raw EEG (EEGNetCNN + Transformer)

EEGNetCNN (modified EEGNet architecture):

Layer	Details
Temporal Conv	Conv2d(1, 64, (1, 64)), padding=same, no bias
BatchNorm	BatchNorm2d(64)
Spatial Conv	Conv2d(64, 256, (22, 1)), groups=64 (depthwise)
BatchNorm + ELU + AvgPool(1,4) + Dropout(0.4)	
Separable Conv	Depthwise Conv2d(256, 256, (1, 16)) + Pointwise Conv2d(256, 256, (1, 1))
BatchNorm + ELU + AvgPool(1,4) + Dropout(0.4)	
Projection	Linear(256, 64) → embed_dim

Output shape: `(B, T', 64)` where `T'` = temporal dimension after pooling

Transformer Encoder:

Component	Config
CLS Token	Learnable parameter, trunc_normal init (std=0.02)
Positional Encoding	Sinusoidal, max_len=300, batch_first=True
Encoder Layers	2 layers
Attention Heads	4 heads

d_model	64
Feed-forward dim	256
Activation	GELU
Dropout	0.2 (transformer), 0.3 (output)

The CLS token output is extracted, layer-normed, and dropped → `raw_feat` (B, 64)

4.3 Branch 2: Spectrogram CNN

Step	Details
STFT	n_fft=128, hop_length=32, Hann window, center=False
Power	<code>log1p(magnitude²)</code>
CNN	Conv2d(22→32, 3×3) → BN → ReLU → MaxPool(2,2) → Conv2d(32→64, 3×3) → BN → ReLU → AdaptiveAvgPool(4,4)
Head	Flatten → Linear(1024, 128) → ReLU → Dropout(0.3)

Output: `spec_feat` (B, 128)

4.4 Branch 3: Band Power MLP

Band	Frequency Range
Delta	0.5–4.0 Hz
Theta	4.0–8.0 Hz
Alpha	8.0–13.0 Hz
Beta	13.0–30.0 Hz
Gamma	30.0–40.0 Hz

Pipeline: `rfft → |spectrum|2 → band_masks → mean per band → log1p → (B, 22×5=110) → Linear(110→128) → BN → ReLU → Dropout(0.3) → Linear(128→64)`

Output: `band_feat` (B, 64)

4.5 Fusion + Classifier

```
fused = cat([raw_feat(64), spec_feat(128), band_feat(64)]) = (B, 256)
→ Linear(256, 128) → ReLU → Dropout(0.3) → Linear(128, 2) → logits
```

4.6 Key Architectural Decisions

- 1. **6 BatchNorm layers** — root cause of cross-dataset probability shift (stores training data statistics); this is why domain shift detection and adaptive post-processing are necessary
- 2. **batch_first=True** throughout — eliminates permute overhead
- 3. **CLS token** for sequence classification — standard Vision Transformer approach
- 4. **Three representation branches** — captures temporal (raw waveform), spectral (STFT), and frequency-band (EEG band power) features simultaneously; no single branch alone achieves competitive performance

5. Training

Source: Notebook cells 36–37 (commented out for safety, preserved for reproducibility)

5.1 V4 Production Configuration

Hyperparameter	Value
Optimizer	AdamW

Learning Rate	3e-4
Weight Decay	1e-3
Epochs	100 (early stopped at epoch 5)
Warmup Epochs	5
Patience	20
Min Delta	0.002
Gradient Clipping	0.5
Mixed Precision	AMP (CUDA)
LR Schedule	Warmup + Cosine Annealing (min 0.01)

5.2 Loss Function: Focal Loss

```
class FocalLoss:
    alpha = [1.0, n0/n1] # Class weights (balanced by count ratio)
    gamma = 2.0 # Focus parameter
    loss = ((1 - pt) ** gamma) * CE
```

Key design choices: - `gamma=2.0` produces cleaner probability curves than `gamma=1.0` - Class weights derived from actual training set count ratio (`n0/n1`) - Focal loss downweights easy examples, focusing on hard seizure boundaries

5.3 Model Version History

Version	Status	Notes
V1	✗ Broken	Alpha bug: <code>alpha=[1.0, 20.13]</code> corrupted class weights
V2	Skipped	Internal numbering skip
V3	✗ Broken	Same alpha bug as V1
V4	✓ Production	Fixed alpha, 409K params, best val F1=0.979
V5	✗ Discarded	Larger/deeper model, overfit

V2 was skipped intentionally. V1 → V3 → V4 reflects internal session numbering. V5 was evaluated and discarded.
V4.0.1 is the current production version.

5.4 V4 Training Results

Metric	Value
Best Epoch	5
Best Val Macro F1	0.9790
Test Macro F1	0.9786

5.5 Lesson: Debug Before New Architecture

Sessions 4-5 discovered that V1/V3 had `alpha=[1.0, 20.13]` — the positive class weight was computed as `n0/n1 = 28,821/1,432 ≈ 20.13` (wrong — counted wrong), corrupting training. After fixing this single bug, V4 trained cleanly in 5 epochs. V5 (larger model) was unnecessary and overfit.

6. Evaluation Metrics

Source: Notebook cells 51, 53 + `config.py` `PRODUCTION_METRICS`

6.1 Window-Level Metrics (Test Set)

Metric	At threshold=0.50	Notes
Accuracy	99.55%	Inflated by 16:1 class imbalance
Macro F1	0.979	Primary metric — balances both classes
Sensitivity (TPR)	97.75%	Seizure recall
Specificity (TNR)	99.59%	Non-seizure recall
ROC-AUC	0.980	Threshold-independent
PR-AUC	0.9474	Best for imbalanced data

Confusion Matrix (threshold=0.50, test set):

	Predicted Non-Sz	Predicted Seizure
Actual Non-Sz	22,844 (TN)	96 (FP)
Actual Seizure	138 (FN)	1,567 (TP)

6.2 Why Accuracy Is Misleading

- Test set is ~94.1% non-seizure windows (16:1 imbalance)
- A model that **always predicts non-seizure** gets 94.1% accuracy (naive baseline)
- NeuroSentinel’s 99.55% is only +5.45pp above this baseline
- **Always lead with Macro F1 and PR-AUC as primary metrics**

Metric	Naive Baseline	NeuroSentinel V4	Delta
Accuracy	94.1%	99.55%	+5.45pp
Macro F1	0.485	0.979	+0.494
Sensitivity	0.0%	97.75%	+97.75pp
PR-AUC	0.059	0.9474	+0.888

6.3 Overfitting Diagnostic (Session 19)

Metric	Train	Val	Test	Gap
Macro F1	~0.98	0.979	0.979	<0.01
ROC-AUC	~0.98	~0.98	0.980	<0.01

Verdict: NOT overfit. High accuracy is a class-imbalance artifact, not overfitting. Train/Test F1 gap < 0.02.

6.4 Event-Level Metrics (from FINAL_results.json)

Metric	Value
Event Sensitivity	73.3% (22/30 seizures detected)
Event FP/hour	0.98
Detection Latency	3.5 seconds (median)
Post-processing config	ADP_p99_d15_mm0.9_s5

6.5 Cross-Dataset Metrics (Siena — Zero-Shot)

Metric	Value
Event Sensitivity	80.0% (4/5 seizures with verified GT)
Events/hour	1.67
Files Tested	11
OOD detection threshold	0.58 (median probability baseline)

6.6 Declared Production Metrics

From `config.py` `PRODUCTION_METRICS`:

```
PRODUCTION_METRICS = {
    "version": "V4 production",
    "accuracy_pct": 99.55,
    "macro_f1": 0.979,
    "event_fp_per_hour": 0.98,
    "event_sensitivity_pct": 73.3,
    "detection_latency_seconds": 3.5,
    "zero_shot_siena_sensitivity_pct": 80.0,
    "zero_shot_siena_events_per_hour": 1.67,
}
```

6.7 What to Lead With in Paper

1. Window-level Macro F1: **0.979**
2. ROC-AUC: **0.980**
3. PR-AUC: **0.9474**
4. Event Sensitivity: **73.3%** (22/30 seizures, test patients)
5. Event FP/hour: **0.98**
6. Zero-shot Siena: **80.0% sensitivity / 1.67 FP/h**
7. Model size: **409,154 parameters**

Mention accuracy (99.55%) **once** in the ablation table with footnote: “Accuracy is inflated by the 16:1 class imbalance; Macro F1 and ROC-AUC are the primary evaluation metrics.”

7. Post-Processing Pipeline

Source: `neurosentinel-backend/app/pipeline/inference.py` + notebook cell 35

7.1 Post-Processing Chain

Raw Probabilities → Smoothing → Domain Shift Detection →
Adaptive Thresholding → Hysteresis → Min Duration Filter →
Merge Close Events → Mean Probability Filter → Sustained Filter →
Final Events

7.2 Smoothing

Moving average with configurable window: `kernel = ones(window) / window`

7.3 Domain Shift Detection

Function: `compute_output_domain_shift()`

Detects how different the input data is from training data using **model output statistics** (not signal statistics):

Prob Median	Domain Shift	Label
< 0.05	0.0	NONE (same domain)
0.05–0.15	0.3	LOW
0.15–0.30	0.6	MODERATE
≥ 0.30	0.9	HIGH (foreign data)

[!IMPORTANT] Signal-level statistics (like TTN — test-time normalization) do NOT reliably detect domain shift because IQR normalization already aligns signal statistics. The shift manifests at the **feature level** and is only visible in **model output probabilities**.

7.4 Domain-Adaptive Thresholds (4-Tier System)

Domain Shift	Smooth Win	Min Duration	Min Mean	Sustained Sec	Sustained Thresh	Threshold Mode
HIGH (≥ 0.7)	13	20s	0.88	8s	0.88	Fixed (0.90/0.75)
MODERATE (≥ 0.4)	11	15s	0.80	5s	0.75	Percentile p99
LOW (≥ 0.15)	7	10s	0.65	4s	0.55	Percentile p97
NONE (< 0.15)	5	8s	0.55	3s	0.45	Percentile p95

7.5 Seizure-Aware Fallback

If post-processing kills ALL events but the model clearly detects seizure activity (`raw_max > 0.5` OR `seizure_ratio > 0.05`), **AND** domain shift is MODERATE or HIGH (`output_shift >= 0.4`), the system retries with permissive thresholds (`domain_shift` forced to `0.0`).

[!NOTE] **Session 20 clarification:** When domain shift is already NONE (< 0.15), the seizure-aware fallback is a **no-op** because the system is already at the most permissive tier. This is the exact scenario the SUSPICIOUS diagnostic state handles — the pipeline behavior is correct; the reporting layer describes what happened accurately.

7.6 53-Config Sweep Results (Session 7)

The winning configuration from 53 post-processing configurations:

`ADP_p99_d15_mm0.9_s5` :

```
smooth_window = 9
high_thresh = p99 adaptive
low_thresh = 0.75
min_duration = 15s
merge_gap = 5s
min_mean_prob = 0.90
sustained_sec = 5s
sustained_thresh = 0.85
```

Config	Sensitivity	FP/h	Notes
Static threshold (0.5)	93%	4.2	Too many FPs
Static threshold (0.9)	56%	0.4	Misses seizures
ADP_p99_d15_mm0.9_s5	73.3%	0.98	Winner

7.7 What Failed (Lessons)

Approach	Result	Why
TTN (Test-Time Normalization)	✗ Failed	IQR normalization already aligns signal stats
BNA (BatchNorm Adaptation)	✗ Failed	Increased CHB-MIT prob_mean from 0.001 to 0.78
Probability Calibration	✗ Failed	Touched detection thresholds, broke sensitivity
Static Thresholds	✗ Failed	Cannot span all patients — per-recording baseline needed

8. Session 20: Diagnostic State Machine (V4.0.1 Patch)

8.1 Problem Statement

An external referential EEG file (`ma0844az_1-1+.edf`) exposed a truth-model inconsistency:

- **101 seizure-like windows** detected by the model (max probability 92.7%)
- **Seizure ratio:** 5.6% of all windows
- **Post-processing** removed all discrete events (duration/sustained filters)
- **Final event count:** 0

The system simultaneously reported: - “No Seizure” (result label) - “Seizure-pattern activity detected” (SCOUT) - “Seizure detected with medium risk” (SCOUT summary) - “No Seizure Events Detected” (UI) - “No segments showed concerning patterns” (SCOUT patient narrative)

These were **mutually contradictory** — the root cause was a missing intermediate diagnostic state.

8.2 Architectural Resolution: 3-State Diagnostic State Machine

```
if enriched_events:
    diagnostic_state = "DETECTED"      # Post-processed events exist
elif model_detects_seizure:
    diagnostic_state = "SUSPICIOUS"    # Model saw patterns, PP removed them
else:
    diagnostic_state = "CLEAR"         # Model did not detect seizure activity
```

Model detects seizure = `raw_prob_max > 0.5 OR seizure_ratio > 0.05`

8.3 SUSPICIOUS State Data Structure

```
{
  "n_windows_above_threshold": 101,
  "max_probability": 0.927,
  "seizure_ratio": 0.056,
  "pattern_alert_level": "elevated",
  "filter_criteria": {
    "min_duration_sec": 8.0,
    "min_mean_prob": 0.55,
    "sustained_sec": 3,
    "sustained_threshold": 0.45
  },
  "reason": "Seizure-like probability spikes were detected but did not meet post-processing criteria..."
}
```

8.4 Downstream Impact

Component	DETECTED	SUSPICIOUS	CLEAR
Result label	“Seizure Detected”	“Suspicious Activity”	“No Seizure”
SCOUT opening	“detected seizure-like activity”	“found suspicious patterns — N segments flagged but none met confirmation criteria”	“did not detect any seizure activity”
Email subject	“Seizure Activity Detected”	“Suspicious Patterns Found”	“No Seizure Activity Detected”
PDF §4	Full event table	Candidate window count + max probability	“No seizure events detected”
UI event cards	Event cards with timing	🔍 “No Confirmed Seizure Events” + explanation	✅ “No Seizure Events Detected”
Reliability badge	Pill-style (emerald/amber/red)	Pill-style	Pill-style

9. Comparison with Published Models on CHB-MIT

9.1 Window-Level Performance

Model / Study	Year	Architecture	Accuracy	Sensitivity	F1	AUC	Params
NeuroSentinel V4	2026	CNN+Transformer+BandPower	99.55%	97.75%	0.979	0.980	409K
EEGNet (Lawhern)	2018	Depthwise Separable CNN	~93%	~88%	~0.85	~0.94	~2.6K
CNN-LSTM Hybrid	2023	Conv + BiLSTM	97.2%	96.1%	0.92	0.97	~1.2M
CNN-Transformer	2024	Conv + Multi-head Attention	98.1%	97.0%	0.95	0.98	~800K
Patient-Specific DL	2024	Per-patient fine-tuned CNN	99.1%	97-100%	0.96	0.99	~500K
SzCORE Benchmark (strict)	2024	Various	—	—	~0.43	—	—

[!IMPORTANT] Direct comparison is challenging because studies differ in: (a) patient-specific vs cross-patient splits, (b) segment duration, (c) overlap ratio, (d) pre-processing, and (e) evaluation protocol. NeuroSentinel uses **strict patient-level splits** — no patient appears in multiple sets — which is more rigorous than random-segment splitting.

9.2 Event-Level Performance

Model / Study	Sensitivity	FP/hour	Latency	Cross-Dataset	Notes
NeuroSentinel V4	73.3%	0.98	3.5s	80% (Siena)	Patient-independent, zero-shot
Soft Fusion CNN (2025)	92.8%	0.8	—	Not tested	Patient-specific fine-tuning
Patient-Specific DL (2024)	97-100%	0.22-0.40	—	Not tested	Per-patient model
Standard CNN (2023)	82%	2.1	8.2s	Not tested	Cross-patient
SzCORE Strict (2024)	Varies	Varies	—	—	F1≈43%

9.3 Key Differentiators

Feature	NeuroSentinel V4	Typical Published Models
Patient-independent	✔ Yes	Often patient-specific
Cross-dataset (zero-shot)	✔ Tested on Siena	✗ Rarely tested
Domain shift adaptation	✔ Output-based adaptive PP	✗ Not addressed
End-to-end clinical system	✔ Web app + PDF + SCOUT AI	✗ Research notebooks only
Explainability	✔ Channel importance + brain heatmap	✗ Often absent
Diagnostic state transparency	✔ DETECTED/SUSPICIOUS/CLEAR	✗ Binary only
Model size	409K params (lightweight)	500K-1.5M (heavier)
Deployment-ready	✔ HF Spaces free tier	✗ Requires GPU

10. Known Limitations

1. **CPU vs GPU BatchNorm drift:** Non-seizure prob_mean = 0.33 on val_loader (CPU). Does NOT affect detection (thresholds are relative, not absolute).
2. **Siena domain shift:** Background probs 0.35–0.51. Handled by per-recording adaptive thresholds.
3. **PN05 patient family:** 2/11 Siena files show entire recording elevated (OOD). Flagged and skipped.
4. **3 channels always missing on Siena:** FT9/FT10 not in standard 10-20. Zero-filled, compatible.
5. **Peri-ictal FPs:** PN06-5, PN09-1 produce FPs near real seizures. Not fixable without patient metadata.
6. **Pediatric training only:** Trained on CHB-MIT (pediatric). Adult seizure patterns may differ.
7. **RAM ceiling:** Initial `mne.io.read_raw_edf` still loads full file header. EDF files > 200 MB may OOM on 4 GB systems.

11. Lessons Learned (Sessions 1-21)

#	Lesson
1	V1 threshold 0.5 was random — alpha=[1.0, 20.13] bug
2	Debug BEFORE trying new architectures
3	V5 (larger/deeper) overfit; V4 (lighter) was already good enough
4	Focal gamma=2.0 > gamma=1.0 for cleaner probability curves
5	Post-processing amplifies model quality; tune it last
6	NEVER change preprocessing pipeline for inference
7	BatchNorm stores dataset-specific stats → root cause of domain shift
8	TTN/BNA/calibration ALL FAILED — do not attempt again
9	Adapt INPUT to match training format; not the other way around
10	Domain shift is best detected from OUTPUT prob median
11	One fixed threshold cannot span all patients — use per-recording baseline
12	Simple solutions beat complex ones (adapter + strict PP)
13	GT annotation bugs mask real performance (typos, wrong timestamps)
14	OOD detection via baseline prob is honest — flag and skip, don't fake
15	Peri-ictal EEG genuinely resembles seizure — some FPs are clinically reasonable
16	Report generation must have zero external dependencies — never fail at deployment
17	PR-AUC is more honest than ROC-AUC for imbalanced medical data
18	Chunked inference is essential for RAM-constrained deployment
19	Never lead with accuracy on imbalanced medical data — report Macro F1 + PR-AUC
20	Binary truth models break on borderline cases — implement intermediate diagnostic states (SUSPICIOUS) with transparent candidate reporting
21	SCOUT race conditions occur if sendMessage fires before provider state settles — use state-aware auto-send logic, not blind timers

12. Saved Artifacts

Category	Path	Contents
Production Model	model_checkpoints_v4/multirep_best_model_v4.pt	V4 weights (409K params)
Inference Cache	model_checkpoints_v4/step4_inference_cache.json	95 test EDFs results
PP Sweep	model_checkpoints_v4/FINAL_results.json	53 post-processing configs
Metrics Plot	model_checkpoints_v4/roc_metrics_v4.png	ROC + PR + F1-vs-threshold
Honest Metrics	neurosentinel_honest_metrics.png	Train/Val/Test comparison
Preprocessed Data	chbmit_preprocessed/	1312 .npy files
Labels	chbmit_labels/	930 files + metadata CSVs
Legacy Models	model_checkpoints_v3/ , v5/	Broken/discarded
HF Model Hub	manikondaadwith/neurosentinel-v4	Public model weights

Continues in: [Chapter 2 — Backend, Inference & Report Generation](#)

NeuroSentinel AI Guide - Part 2

NeuroSentinel AI — Internal Technical Guide

Chapter 2: Backend Architecture, Inference Pipeline, Report Generation, SCOUT & Email

Purpose: Internal reference document for conference/journal paper and institutional memory.
Last updated: September 2026 (Sessions 21+)
Authors: Manikonda Adwith — KMEC, Hyderabad — G1108

1. Backend Architecture

1.1 Stack

Component	Technology
Framework	FastAPI (Python)
Hosting	Hugging Face Spaces (Docker, free tier)
Model Runtime	PyTorch (CPU only — no GPU on free tier)
Concurrency	<code>threading.Thread</code> (daemon) + <code>threading.Lock</code>
Auth	Supabase JWT verification
Memory Management	cgroup v2/v1 reading + <code>psutil</code> fallback

1.2 Repository Layout

The repository has **two parallel backend directories**:

Path	Purpose
<code>/app/</code>	Root-level directory for local development and testing
<code>/neurosentinel-backend/app/</code>	Containerized deployment package — this is what runs on Hugging Face Spaces

Both directories share the **exact same module structure**:

```
app/
├── main.py                # FastAPI application factory
├── config.py              # Global config (env vars, production metrics)
├── api/
│   ├── deps.py           # Supabase auth dependency injection
│   └── schemas.py         # Pydantic request/response schemas
├── pipeline/
│   ├── config.py         # Pipeline-specific constants
│   ├── model.py           # MultiRepEEGModel definition
│   ├── preprocessing.py   # Full preprocessing pipeline
│   ├── inference.py       # Inference + post-processing (consolidated)
│   ├── explainability.py  # Channel importance + brain heatmap
│   └── reporting.py       # Clinical report builder (diagnostic state machine)
├── services/
│   ├── analysis_service.py # Pipeline orchestration
│   ├── email.py           # Multi-provider email with 3-way state logic
│   ├── pdf.py             # ReportLab PDF generation (~1348 lines)
│   ├── supabase.py        # SupabaseService with resilient schema fallback
│   └── scout/
│       ├── service.py     # SCOUT AI assistant (~1481 lines)
│       └── providers.py   # LLM provider fallback chain
```

[!NOTE] neurosentinel-backend/ also contains a tests/ folder with 4 test modules: test_chunked_inference.py , test_domain_shift.py , test_full_pipeline_mocked.py , and test_reporting.py .

1.3 Application Factory (main.py)

create_app() builds the FastAPI instance with:

- **Lifespan manager** — initializes BackendState , marks any leftover processing jobs as failed on startup (stale job recovery)
- **Strict inference lock** — threading.Lock() ensures only ONE analysis runs at any time (protects 2 GB RAM ceiling on free tier)
- **Lazy model loading** — model loads on first inference request, not at startup (reduces cold-start time)
- **CORS** — allows all origins (Vercel frontend)

1.4 API Endpoints

Endpoint	Method	Auth Required	Purpose
/health	GET	No	Service status, memory, model readiness
/api/v1/model/info	GET	No	Model version, parameters, declared metrics
/api/v1/job/status	GET	Bearer	Current job status, elapsed time, memory
/api/v1/job/cancel	POST	Bearer	Cancel running analysis
/api/v1/analyze	POST	Bearer	Direct EDF upload → analysis
/api/v1/analyze-url	POST	Bearer	Supabase Storage URL → analysis
/api/v1/scout/chat	POST	Bearer	SCOUT AI conversation
/api/v1/internal/send-otp-email	POST	X-Internal-Secret	OTP email relay from HF backend
/api/v1/debug/config	GET	No	Debug deployment configuration flags

1.5 Threading Model



1.6 Guard Function

Called at every chunk and every batch boundary during inference. Checks:

1. **Cancel** — user requested abort via `/api/v1/job/cancel`
2. **Timeout** — elapsed time exceeds `JOB_TIMEOUT_SECONDS = 3600` (1 hour)
3. **Memory** — reads from cgroup v2 (`/sys/fs/cgroup/memory.stat`) or cgroup v1 (`memory.usage_in_bytes`), falls back to `psutil.Process().memory_info().rss` . Logs warning and triggers `gc.collect()` if `RSS > 450 MB`

1.7 Upload Architecture (Proxy Pattern)

The frontend does **NOT** route large `.edf` files through Vercel (which has a 4 MB body limit on serverless functions). Instead:



This avoids Vercel serverless body size limits and keeps the upload path clean.

2. Inference Pipeline (Production)

2.1 Two Inference Paths

Function	Use Case	Input
<code>infer_from_data_chunked()</code>	Large files, production	Continuous <code>(22, n_samples)</code> array
<code>analyze_preprocessed_windows()</code>	Legacy/testing	Pre-windowed <code>(n_windows, 22, 1024)</code>

2.2 Chunked Inference (Primary Path)

Designed specifically for Hugging Face Spaces free-tier constraints:

Parameter	Value	Reason
<code>max_windows_per_chunk</code>	50	~4 MB per chunk at float32
<code>batch_size</code> (adaptive)	1-4	Based on total windows
Guard checks	Every chunk + every batch	Memory/cancel/timeout safety

Adaptive batch sizing (`_resolve_chunk_batch_size`):

Total Windows	Batch Size
≥ 1500	1
≥ 800	2
≥ 300	2
< 300	4

2.3 Memory Discipline

- Windows extracted using stride tricks (view, not copy)
- Each chunk's tensors `del`-ed immediately after inference
- `gc.collect()` called after every chunk boundary
- Representative window (highest seizure probability) tracked for explainability without caching all tensors
- Quality samples capped at 10 windows

2.4 Model Weight Loading

- Weights stored in Supabase Storage bucket `ml-models`
- Downloaded once via streaming HTTP to `/tmp/multirep_best_model_v4.pt`
- Thread-safe singleton loading via `_model_lock`
- Loaded to CPU (no GPU on Hugging Face Spaces free tier)
- Model hosted publicly: `manikondaadwith/neurosentinel-v4` on HuggingFace Model Hub

2.5 Production Configuration

Setting	Value	Reason
<code>analysis_batch_size</code>	4 (adaptive)	Ultra-low for HF Spaces RAM
<code>scout_max_history</code>	20 messages	Limits LLM context window
<code>scout_report_limit</code>	8 reports	Max recent reports for SCOUT
<code>backend_timeout_seconds</code>	60	HTTP request timeout (frontend)
<code>JOB_TIMEOUT_SECONDS</code>	3600	Max inference runtime
<code>MAX_UPLOAD_BYTES</code>	1 GB	Maximum .edf file size
<code>MEMORY_THRESHOLD_MB</code>	450	<code>gc.collect()</code> trigger threshold

3. Report Generation (Session 20 — V4.0.1)

Source: `app/pipeline/reporting.py` (~605 lines)

3.1 Diagnostic State Machine

The canonical 3-state truth model (implemented in Session 20):

```
if enriched_events:
    diagnostic_state = "DETECTED"    # Confirmed seizure events
elif raw_prob_max > 0.5 or seizure_ratio > 0.05:
    diagnostic_state = "SUSPICIOUS"  # Model detected patterns; PP filtered them
else:
    diagnostic_state = "CLEAR"        # No seizure activity
```

This is the **single source of truth** propagated to all downstream layers: SCOUT, email, PDF, recommendations, UI cards.

3.2 Clinical Report JSON Structure

`build_clinical_report()` produces nested JSON with:

```
{
  "meta": {
    "recording_id": "...",
    "generated_at": "ISO-8601",
    "duration_min": 45.2,
    "tool": "NeuroSentinel AI V4.0.1",
    "disclaimer": "..."
  },
  "signal_quality": {
    "grade": "High",
    "score": 0.87,
    "missing_channels": []
  },
  "summary": {
    "total_events": 3,
    "overall_risk": "High",
    "trend": "Worsening",
    "early_warning": false,
    "se_flag": false
  },
}
```

```

    "events": [
      {
        "onset": 124.0,
        "offset": 142.0,
        "duration": 18.0,
        "confidence": 0.94,
        "severity": "High",
        "risk": "Critical"
      }
    ],
    "explainability": {
      "top_channels": ["FP1-F7", "FP2-F8"],
      "top_regions": ["Left Temporal", "Right Temporal"]
    },
    "recommendations": [...],
    "diagnostic_state": "DETECTED",
    "suppressed_candidates": null,
    "result_label": "Seizure Detected"
  }

```

For `SUSPICIOUS` state: `result_label` = "Suspicious Activity", `suppressed_candidates` contains `n_windows_above_threshold`, `max_probability`, `seizure_ratio`, `pattern_alert_level`, `filter_criteria`, `reason`.

3.3 Clinical Heuristics

- **Recording Quality:** Mean quality score based on SNR (dB), line noise ratio, and flatline fraction
- **Seizure Type Characterization:** Rule-based classification:
 - Focal Impaired Awareness (short duration, isolated channels)
 - Focal Motor (asymmetric channel spread)
 - Generalized Clonic, Generalized Tonic-Clonic (bilateral spread)
 - Unclassified (default)
- **Early Warning flag:** Probability ramp detected across 3 windows preceding event onset
- **Status Epilepticus flag:** Single event ≥ 300 seconds OR cumulative seizure burden $\geq 50\%$ in any 10-minute window

3.4 Suspicious Risk Derivation

`_derive_suspicious_risk()` scores evidence burden 0-10:

Factor	Max Points	Formula
Window count	3.0	<code>min(n / 20, 1.0) × 3</code>
Peak probability	3.0	<code>raw_max × 3</code>
Seizure ratio	2.0	<code>min(ratio × 30, 1.0) × 2</code>
Poor quality penalty	-1.5	Applied if quality == "poor"
High domain shift (≥ 0.7)	-1.0	Applied if output_shift HIGH
Moderate domain shift (≥ 0.4)	-0.5	Applied if output_shift MODERATE

Score	Risk Level	Alert Level
≥ 5.0	Medium	Elevated
≥ 2.0	Medium	Low
< 2.0	Low	Low

3.5 Recommendations Engine

`_build_recommendations()` accepts `diagnostic_state` and generates role-appropriate recommendations:

- Status epilepticus: urgent neurologist review within 24h
- Critical/High risk: escalate immediately
- **SUSPICIOUS state:** Neurologist review to evaluate clinical significance; consider extended or repeat EEG monitoring
- Early warning: close patient monitoring

- Multiple events: evaluate for seizure clustering
- Always: correlate with clinical observation

4. Explainability

Source: `app/pipeline/explainability.py`

4.1 Channel Importance (Perturbation Method)

Perturbation-based approach (saves 80–150 MB vs gradient methods):

1. Get baseline seizure probability from representative window
2. For each of 22 channels: zero out, re-run inference, measure probability drop
3. Importance = `max(0, baseline_prob - perturbed_prob)`
4. Normalize: `importance / sum(importance)` → sums to 1.0

4.2 Attention Maps

`extract_attention_maps()` patches Transformer `MultiheadAttention.forward`. Extracts CLS-to-token attention weights from the last encoder layer.

4.3 Brain Heatmap

`generate_brain_heatmap()` : - Maps 22-channel importances to standard 10-20 electrode positions - Region scores = mean of constituent channel importances - Returns a `{region: score}` dict used by the `BrainHeatmap` React component

5. PDF Generation (ReportLab)

Source: `app/services/pdf.py` (~1348 lines)

5.1 Technology

ReportLab canvas API, Letter-size pages (8.5" × 11"), clinical teal color palette with NeuroSentinel branding.

5.2 PDF Sections

Section	Content
Cover / Header	NeuroSentinel AI logo (base64-embedded), report metadata, recording ID
S0	Report Information (patient, date, file, clinician)
S1	Signal Quality (grade, score, missing channels, visual quality bar)
S2	Executive Summary (result label, risk, confidence, duration, event count)
S3	SCOUT Narrative (role-aware text summary from SCOUT)
S4	Events Table — state-aware (DETECTED: event table; SUSPICIOUS: candidate count + max prob; CLEAR: clean bill)
S5	Channel Attribution (top channels ranked by perturbation importance)
S6	Recommendations — state-aware (DETECTED: urgent/share; SUSPICIOUS: extended EEG; CLEAR: continue monitoring)
S7	Pipeline Metadata (model version, post-processing config, domain shift score)
Footer	Clinical disclaimer on every page

5.3 Post-April Updates to PDF

After April 21, 2026:

- **Charts added:** Probability Timeline chart and Band Power chart are now rendered using Matplotlib and embedded as base64 PNG images in the PDF
- **Logo aligned:** NeuroSentinel AI logo (logo.jpeg) is now embedded adjacent to the branding header
- **Teal clinical theme:** Header and section bars use clinical teal (#0EA5A4) instead of blue
- **Download button:** “Open Report PDF” opens in a new browser tab (not forced download) to avoid popup blockers
- **PDF lib change:** The PDF is generated by the **Python backend** (ReportLab), proxied via `/api/reports/[id]/pdf` (Next.js route) which fetches from Supabase Storage `report-pdfs` bucket path `{user_id}/{report_id}.pdf`

5.4 Role-Aware Report Tone

Role	Tone	Content Focus
Patient	Warm, paragraph-based	What happened, what to do next, health tips
Clinician	Metric-dense, structured	Lateralization, onset hypothesis, SE protocol
Researcher	Technical, methodological	Pipeline params, probability distribution, domain shift

6. SCOUT AI Assistant

Source: `app/services/scout/service.py` (~1481 lines)

6.1 What SCOUT Is

SCOUT = Seizure Clinical Operations and Understanding Tool. Role-aware, context-aware AI assistant embedded in every page of the NeuroSentinel frontend. Unlike a generic LLM chat, SCOUT has:

- Full access to the current report’s JSON
- Up to 8 recent reports for trend comparison
- User role and preferences from Supabase
- Page context (dashboard stats, active uploads)
- Built-in clinical knowledge base (no external dependency)
- Architectural privacy guardrails (cannot leak infrastructure details)

6.2 LLM Provider Fallback Chain

Priority	Provider	Model	Trigger
1	Google Gemini	<code>gemini-2.5-flash</code>	Primary
2	Groq	<code>llama-3.3-70b-versatile</code>	If Gemini fails/times out
3	Hugging Face	<code>Llama-3.2-3B-Instruct</code>	Last resort
4	Deterministic	Built-in rule-based generator	If all LLMs fail

6.3 Role-Aware Tone

Role	Tone Rules
Clinician	Precise, clinical, metric-dense. Structured output. Short follow-ups.
Researcher	Technical, methodological. Narrative with embedded metrics.
Patient	Warm, calm, conversational. Paragraphs only — never numbered lists, never bullet points.

6.4 Context Assembly

For each chat request, SCOUT assembles:

1. System prompt: role, tone rules, guardrails, diagnostic state grounding
2. Report context: full `report_json` parsed into structured details
3. Recent reports: up to 8 prior reports for trend comparison
4. User profile: email, name, preferences
5. Page data: dashboard stats, active uploads
6. Product guide snippets: relevant help content
7. Session history: last 20 messages

6.5 Diagnostic-State-Aware Summaries (Session 20)

Patient summary branches on `diagnostic_state`:

State	Patient Opening Summary
DETECTED	"I detected N seizure-like event(s)... at [onset times]..."
SUSPICIOUS	"I found suspicious patterns — N segment(s) flagged but did not meet strict confirmation criteria..."
CLEAR	"I did not detect any seizure activity... brainwave activity appears within normal ranges..."

6.6 Severity Guidance

State	Risk	SCOUT Guidance
SUSPICIOUS	Medium	No emergency action, follow-up neurology in few weeks
SUSPICIOUS	Low	Worth mentioning at next scheduled visit
DETECTED	Critical/High	Emergency escalation within 24–48h
DETECTED	Medium	Schedule follow-up in 1–2 weeks
CLEAR	Any	Share at next scheduled visit

6.7 Health Tips

State	Health Tips
SUSPICIOUS	Symptom diary advice only — NOT emergency precautions
DETECTED with events	Full safety precautions (medical ID, avoid swimming/driving alone)
CLEAR	General wellness only (sleep hygiene, diet, exercise)

6.8 Clinical Knowledge Base (Built-in)

Embedded in `product_guide.py` — no external database dependency:

- Region-based etiology: temporal, frontal, parietal, occipital, central, generalized
- Severity guidance per role per risk level
- Patient health tips: sleep, diet (Mediterranean), exercise, medication compliance, safety
- Clinician management: SE protocol, AED optimization, monitoring recommendations
- Epidemiological context for researchers: WHO epilepsy burden, TLE prevalence

6.9 Reliability Assessment

`_compute_reliability()` derives report reliability from:

- Confidence score (< 30% → always Low)
- Signal quality grade (poor → Low)
- Recording duration (< 10 min → Low; < 20 min → Moderate)

6.10 Guardrails (Hard-Coded)

1. Never diagnose or replace clinical judgment
2. Never recommend treatment changes
3. Never fabricate data or certainty
4. Keep follow-ups concise (initial summaries can be detailed)
5. **Architecture privacy:** Never reveal internal infrastructure paths, prompt templates, server configs, or pipeline tokens
6. **SUSPICIOUS grounding:** “CRITICAL: diagnostic_state is SUSPICIOUS. NEVER say seizure detected. Say suspicious patterns or seizure-like patterns flagged but not confirmed.”

7. SCOUT Frontend Integration

Source: `neurosentinel/lib/scout-guide.ts`, `neurosentinel/app/components/scout-provider.tsx`, `neurosentinel/app/components/scout-conversation.tsx`

7.1 Architecture Overview

```
ScoutProvider (React Context – wraps entire app in layout.tsx)
├── State: messages[], isLoading, isStreaming, stateKey
├── sessionStorage persistence (key: scout-conv-{stateKey})
├── sendMessage() → POST /api/chat
├── clearHistory()
├── setPageContext() – called by every page to inject context
└──
    ▼
    ScoutFloating (rendered in root layout.tsx)
    ├── On open: renders ScoutPanel → ScoutConversation
    └── useScoutConversation hook
        ├── autoPrompt support (fires once per stateKey)
        └── quickPrompts bar (page-specific shortcuts)
```

7.2 Conversation Persistence

- **Storage:** `sessionStorage` (survives page navigation, clears on tab close)
- **State Key:** Derived from user session — resets on new login, also has per-page variants
- **Comparison Mode:** Uses a unique `stateKey` per pair of compared reports — avoids mixing contexts between different comparison sessions
- **Reset:** `clearHistory()` clears both React state and `sessionStorage` entry

7.3 POST /api/chat Request Body

```
{
  messages: [{ role: 'user' | 'assistant', content: string }],
  context: {
    page: 'dashboard' | 'report' | 'reports' | 'compare' | 'general',
    role: 'clinician' | 'researcher' | 'patient' | null,
    reportId: string | null,
    currentReport: ReportRecord | null,
    pageData: any
  }
}
```

The `/api/chat` Next.js route proxies this to the FastAPI backend `/api/v1/scout/chat`.

7.4 autoPrompt (Comparison Mode Fix — Session 21)

The comparison page passes an `autoPrompt` object to `ScoutConversation`:

```
autoPrompt={{ content: buildComparisonPrompt(reportA, reportB, role), visible: false }}
```

The `useScoutConversation` hook fires `sendMessage(autoPrompt.content)` exactly **once** per `stateKey`, guarded by:

1. Both reports fully loaded (non-null)
2. No existing messages for this `stateKey` (prevents double-fire on re-render)
3. `useEffect` dependency array includes `[stateKey, reportsReady]` — not a blind timer

[!NOTE] **Session 21 fix:** The original implementation used a blind `setTimeout(600ms)` which caused race conditions — the auto-send fired before the report data was ready, resulting in an empty SCOUT summary. The fix uses a `state-aware` `useEffect` that checks `reportsReady` state before sending. This eliminated comparison SCOUT failures.

7.5 ScoutConversation Component Props

```
interface ScoutConversationProps {
  page: 'dashboard' | 'report' | 'reports' | 'general'
  role: ScoutRole
  reportId?: string | null
  currentReport?: ReportRecord | null
  pageData?: any
  stateKey?: string
  initialMessage?: string
  quickPrompts?: string[]
  autoPrompt?: { content: string; visible: boolean }
}
```

8. Email System

Source: `app/services/email.py` (backend), `neurosentinel/app/api/internal/send-report/route.ts` (frontend relay)

8.1 Provider Priority Chain (Backend)

Priority	Provider	Config
1	Resend API	<code>RESEND_API_KEY</code>
2	Relay Proxy (Vercel)	<code>RELAY_API_URL</code> + <code>INTERNAL_API_SECRET</code>
3	SMTP (Gmail)	<code>SMTP_HOST/PORT/USER/PASSWORD</code>

8.2 Why Resend (Not SMTP from Backend)

SMTP ports 465/587 are **blocked on Hugging Face Spaces** free tier due to network restrictions. Resend API uses HTTPS — works from any host. SMTP is used as a tertiary fallback via the Vercel relay proxy.

8.3 OTP Email Provider Chain (Frontend)

The signup/login OTP flow uses a separate 4-tier dispatch from the Vercel edge (`lib/signup-otp-store.ts`):

Tier	Provider	Notes
1	Direct Gmail SMTP via <code>nodemailer</code> from Vercel	Primary (works if port 587 open from Vercel)
2	Brevo API (<code>api.brevo.com</code>)	Free SMTP API, no domain required
3	Resend API (<code>api.resend.com</code>)	HTTPS-based
4	HF Backend relay (<code>/api/v1/internal/send-otp-email</code>)	Final fallback

8.4 OTP Security

- **Stateless verification:** HMAC-SHA256 tokens encoding `{email, otpHash, expiresAt}` signed with `OTP_SIGNING_SECRET`
- **Existing account check:** Validates against both `public.users` and `auth.users` via Supabase admin client before

sending OTP (prevents sync errors)

- **Timeout chain:** SMTP 15s → fetch 55s → Vercel function 60s max duration

8.5 Report Email Content

HTML emails with NeuroSentinel branding (clinical white theme):

- Report link: `https://neuro-sentinel-ai-6vfv.vercel.app/report/{id}`
- Result summary, risk level, confidence, quality
- Clinical disclaimer
- **CAN-SPAM compliance** (added post-April):
 - `List-Unsubscribe` headers
 - Plain-text fallback
 - Physical address footer: **Champapet, Hyderabad 500079**

8.6 Email Subject Lines (State-Aware — Session 20)

State	Subject Line
DETECTED	Seizure Activity Detected in {filename}
SUSPICIOUS	Suspicious Patterns Found in {filename}
CLEAR	No Seizure Activity Detected in {filename}

8.7 Why Email Deliverability Required These Changes

The original implementation (SMTP only) was flagged as spam by Gmail and Yahoo filters because:

1. **No SPF/DKIM/DMARC** — the `smtp.gmail.com` relay meant emails came from Google’s IP but claimed to be from NeuroSentinel’s domain
2. **Spammy content** — medical words (“seizure”, “detected”) in subject lines without proper authentication signal
3. **No physical address** — CAN-SPAM requires a postal address in commercial emails
4. **No unsubscribe mechanism** — required by modern email standards

Fixes applied:

- Switched primary send path to Resend API (which handles SPF/DKIM automatically on their sending domain)
- Added `List-Unsubscribe` headers (both `mailto:` and HTTPS methods)
- Added physical address footer
- Added plain-text fallback (spam filters penalize HTML-only emails)
- Added `X-Mailer`, `Precedence: bulk` headers

9. Supabase Integration

9.1 Database Tables

Table	Key Columns	Purpose
<code>users</code>	<code>id</code> , <code>email</code> , <code>role</code> , <code>onboarding_complete</code> , <code>preferences</code> , <code>full_name</code>	User profiles
<code>reports</code>	<code>id</code> , <code>user_id</code> , <code>filename</code> , <code>status</code> , <code>report_json</code> , <code>summary</code> , <code>result_label</code> , <code>event_count</code> , <code>confidence_score</code> , <code>quality_grade</code> , <code>risk_level</code> , <code>duration_minutes</code>	Analysis reports
<code>chat_messages</code>	<code>id</code> , <code>user_id</code> , <code>role</code> , <code>content</code> , <code>report_id</code> , <code>page_context</code> , <code>metadata</code>	SCOUT conversations

9.2 Storage Buckets

Bucket	Content
--------	---------

ml-models	Production model weights (multirep_best_model_v4.pt)
report-pdfs	Generated PDF reports ({user_id}/{report_id}.pdf)
edf-uploads	Temporary EDF files uploaded by browser

9.3 Auth Flow

1. Frontend uses Supabase Auth (email/password + OTP for signup)
2. Access token sent as `Authorization: Bearer <token>` header
3. Backend verifies via `GET /auth/v1/user` with service key
4. Returns `AuthenticatedUser(id, email)`

9.4 Resilient Schema Pattern

`SupabaseService` uses try/except fallback:

1. First attempt: full column set (extended schema with all optional columns)
2. Fallback: base columns only (`id, user_id, filename, status, report_json`)

This prevents crashes if extended columns haven't been added yet in a new environment.

9.5 Row-Level Security (RLS)

RLS is enabled on the `reports` table. Users can only read/write their own reports (`user_id = auth.uid()`). The `chat_messages` table similarly scopes to `user_id`.

10. Backend Environment Variables

Variable	Required	Purpose
SUPABASE_URL	✓	Supabase project URL
SUPABASE_SERVICE_KEY	✓	Supabase service role key (bypasses RLS)
GEMINI_API_KEY	✗	Primary LLM for SCOUT
GROQ_API_KEY	✗	Fallback LLM for SCOUT
RESEND_API_KEY	✗	Email notifications
SMTP_HOST/PORT/USER/PASSWORD	✗	Fallback email
APP_URL	✗	Frontend URL for email links
INTERNAL_API_SECRET	🔒	Shared secret for OTP relay
MODEL_PATH	Optional	Local model path override

Continues in: [Chapter 3 — Frontend Architecture & UI](#)

NeuroSentinel AI Guide - Part 3

NeuroSentinel AI — Internal Technical Guide

Chapter 3: Frontend Architecture, UI Design & Deployment

Purpose: Internal reference document for conference/journal paper and institutional memory.

Last updated: September 2026 (Sessions 21+)

Authors: Manikonda Adwith — KMEC, Hyderabad — G1108

1. Frontend Stack

Component	Technology
Framework	Next.js 14 (App Router)
Hosting	Vercel (free tier)
Styling	Vanilla CSS (clinical white theme) + Google Fonts
Auth	Supabase Auth (email/password + OTP)
Global State	React Context (ScoutProvider , AnalysisProvider)
Charts	Recharts (probability timeline, band power chart)
File Upload	react-dropzone
Icons	lucide-react
PDF	pdf-lib (frontend overlay) + ReportLab (backend generation)

1.1 Font Stack

Loaded via Google Fonts in `app/layout.tsx` :

- **Outfit** (400–800): Branding, headings, logo text
- **Inter** (300–700): Body text, navigation, clinical copy
- **JetBrains Mono** (400–600): Metrics, codes, data values

2. Application Routing

All routes are in `neurosentinel/app/` using Next.js 14 App Router:

Route	Type	Purpose
<code>/</code>	Public	Landing page (redirects to <code>/login</code> or <code>/dashboard</code>)
<code>/login</code>	Public	Login + Signup combined page
<code>/onboarding</code>	Protected	SCOUT-led first-run experience (role + name collection)
<code>/dashboard</code>	Protected	Analysis Station — upload zone + recent reports
<code>/dashboard/eeg-reports</code>	Protected	Full analysis history with filters + comparison
<code>/dashboard/eeg-reports/compare</code>	Protected	Side-by-side report comparison + SCOUT summary

/dashboard/settings	Protected	Profile, role preference, password, account management
/dashboard/patient-records	Protected	Redirects to /dashboard/eeg-reports
/report/[id]	Protected	Full clinical report viewer (tabs: clinical, signals, raw)
/terms	Public	Terms of Use
/privacy	Public	Privacy Policy

2.1 Middleware & Auth Guard

The middleware does a **raw cookie check only** (no network calls) to avoid cold-start latency and circular redirect loops with expired cookies. Protected routes check Supabase auth in the layout server component and redirect to / if no session.

2.2 Route Protection Flow



3. Component Architecture

3.1 File Structure (app/)

app/	
├─ layout.tsx	# Root layout – ScoutProvider, ReportNotificationProvider, ScoutFloating
├─ globals.css	# Global CSS reset and base tokens
├─ page.tsx	# Landing page
├─ login/page.tsx	# Combined login + signup
├─ onboarding/page.tsx	# SCOUT-led onboarding flow
├─ dashboard/	
│ ├─ layout.tsx	# Server layout – auth check, sidebar, AnalysisProvider
│ ├─ page.tsx	# Analysis Station (upload + recent reports)
│ ├─ dashboard.css	# Clinical CSS design system (CSS variables)
│ └─ eeg-reports/	
│ │ ├─ page.tsx	# Analysis History (filters, sort, comparison select)
│ │ ├─ layout.tsx	# EEG reports sub-layout
│ │ └─ compare/page.tsx	# Side-by-side comparison + SCOUT
│ └─ settings/	
│ │ ├─ page.tsx	# Server shell
│ │ └─ settings-client.tsx	# Client – name edit, role, password, delete account
│ └─ _components/	
│ │ ├─ sidebar.tsx	# Navigation sidebar (desktop + mobile drawer)
│ │ ├─ upload-zone.tsx	# Drag-and-drop EDF upload
│ │ ├─ analysis-results.tsx	# Result display after processing
│ │ └─ processing-pipeline-card.tsx	# Live processing status card
│ │ └─ reliability-badge.tsx	# Pill-style High/Moderate/Low badge
│ │ └─ status-badge.tsx	# completed/processing/failed badge
│ │ └─ skeleton.tsx	# Shimmer skeleton loaders
│ │ └─ name-prompt.tsx	# Post-onboarding name collection prompt
│ │ └─ scout-tour.tsx	# Guided clinical tour component
└─ report/	
│ └─ [id]/page.tsx	# Full clinical report viewer
│ └─ _components/	
│ │ └─ clinical-metrics.tsx	# Key metrics strip (confidence, risk, quality, events)

```

├── probability-timeline.tsx # Recharts seizure probability timeline
├── brain-heatmap.tsx       # SVG brain heatmap (10-20 electrode positions)
├── event-cards.tsx        # Individual seizure event detail cards
├── data-quality-panel.tsx  # Signal quality breakdown
├── band-power-chart.tsx   # Recharts EEG band power bar chart
└── components/
    ├── scout-provider.tsx  # Global ScoutContext (sessionStorage persistence)
    ├── scout-conversation.tsx # Full chat UI (messages, input, quickPrompts)
    ├── scout-floating.tsx  # Floating launcher button (bottom-right corner)
    ├── scout-panel.tsx     # Panel wrapper (slide-in overlay)
    ├── scout-avatar.tsx    # SCOUT avatar icon component
    ├── report-notification-provider.tsx # In-app toast for completed reports
    └── report-notification-toast.tsx # Toast component

```

3.2 Global Context Providers (Root Layout)

```

// app/layout.tsx
<ScoutProvider>
  <ReportNotificationProvider>
    {children}
    <ScoutFloating /> ← rendered at root, visible on all pages
  </ReportNotificationProvider>
</ScoutProvider>

```

3.3 Dashboard Context Providers

```

// app/dashboard/layout.tsx
<AnalysisProvider>
  <div className="clinical-dashboard-layout">
    <Sidebar userEmail={user.email} />
    <div className="clinical-main-content">
      {children}
    </div>
  </div>
</AnalysisProvider>

```

`AnalysisProvider` holds centralized upload/processing state shared between the upload zone, sidebar status indicator, and analysis results panel. Eliminates prop-drilling across the dashboard.

4. Key Pages — Internal Logic

4.1 Dashboard Page (/dashboard)

State managed by `useAnalysis()` context:

State	Type	Purpose
<code>currentReport</code>	<code>ReportRecord null</code>	Most recent completed report
<code>activeJobId</code>	<code>string null</code>	In-flight job identifier
<code>activeFilename</code>	<code>string null</code>	Filename of uploading EDF
<code>jobStatus</code>	<code>string</code>	processing / completed / failed
<code>uploadProgress</code>	<code>number</code>	0-100 upload percentage
<code>isProcessing</code>	<code>boolean</code>	True while backend analysis runs

Job polling logic: - `setInterval(3000ms)` polls `/api/job-status` - On `status === 'completed'`: fetches report from Supabase, sets `currentReport`, clears interval - On `status === 'failed'`: sets `jobStatus = 'failed'`, clears interval - Backend offline (502/503): sets `backendOffline` state → UI shows a recovery card - Interval cleared on component unmount and on completion

SCOUT context registration:

```

setPageContext({
  page: 'dashboard',
  role: userRole,
  reportId: currentReport?.id ?? null,
  currentReport,
  pageData: { stats, recentReportsCount }
})

```

4.2 EEG Reports Page (/dashboard/eeg-reports)

Key features: - Full Supabase query: all user reports ordered by `created_at DESC` - Client-side filtering (`useMemo`): search by filename/ID, status filter (all/completed/processing/failed), result filter (all/seizure/normal/uncertain), multi-field sort - **Comparison selection:** Toggle up to 2 reports → “Compare Reports” button appears → navigates to `/dashboard/eeg-reports/compare?a={id}&b={id}` - **PDF download:** Fetches from `/api/reports/{id}/pdf` → creates blob URL → triggers `<a>` download

4.3 Compare Page (/dashboard/eeg-reports/compare)

- Reads `?a=` and `?b=` query params
- Fetches both reports from Supabase
- Renders side-by-side metrics table
- Builds a structured `buildComparisonPrompt(a, b, role)` and passes as `autoPrompt` to `ScoutConversation`
- Uses unique `stateKey = \ compare-`${a,b}.sort().join('-')}`` so each pair has independent chat history

Comparison metrics shown: - Result label, risk level, confidence %, duration, event count - Signal quality grade - Domain shift score (if available in `report_json`) - Created-at timestamps

4.4 Report Detail Page (/report/[id])

Three-tab layout:

Tab	Content
Clinical	ClinicalMetrics strip, EventCards, DataQualityPanel
Signals	ProbabilityTimeline, BrainHeatmap, BandPowerChart
Raw	Raw report JSON viewer (for researchers)

ProbabilityTimeline (Recharts): - Renders seizure probability (0–1) over time in seconds - Uses `report_json.proBABILITIES` array - Seizure events overlaid as shaded regions (`ReferenceArea`) - Annotations show event onset/offset with confidence labels

BrainHeatmap: - SVG-based rendering of standard 10-20 electrode positions - Channel importance scores from `report_json.explainability.top_channels` - Colour scale: low importance (light grey) → high importance (deep teal)

BandPowerChart (Recharts): - Bar chart: Delta, Theta, Alpha, Beta, Gamma - Per-channel or averaged, from `report_json.signal_quality`

PDF download handler:

```

const res = await fetch(`/api/reports/${report.id}/pdf`)
const blob = await res.blob()
// Creates blob URL → opens in new tab (not forced download)

```

Report polling: - If report status is `processing` or `pending` on load, polls Supabase every 3s until complete

SCOUT panel: - Fixed-width right panel, toggled via a teal button at bottom-right - Uses `ScoutConversation` with `page='report'`, `reportId`, `currentReport`, `pageData=reportJson` - Quick prompts: “Summarize findings”, “Explain confidence”, “What should I do next?”

5. Sidebar & Navigation

5.1 Navigation Items

Label	Route	Icon
Analysis Station	/dashboard	Dashboard grid
Analysis History	/dashboard/eeg-reports	File document
Settings & Security	/dashboard/settings	Gear
SCOUT Assistant	(opens floating panel)	SCOUT avatar + pulse dot

5.2 Mobile Responsive Drawer (Post-April)

The sidebar gained full mobile support after April 2026:

- **Mobile Header Bar:** Visible only on $\leq 768\text{px}$. Shows hamburger button, NeuroSentinel AI logo, and role pill
- **Drawer overlay:** Sidebar slides in from left (`transform: translateX(-100%)` $\rightarrow$ `translateX(0)`) when `mobileOpen = true`
- **Backdrop:** Semi-transparent overlay closes sidebar on tap
- **Nav link click:** Also closes sidebar (`onClick={() => setMobileOpen(false)}`)
- **CSS breakpoint:** `@media (max-width: 768px)` hides desktop sidebar, shows mobile header bar

5.3 Sidebar Footer

- Active analysis indicator (pulsing dot + filename when `isProcessing`)
- SCOUT status item (shows “SCOUT Active” dot when panel open)
- Help & Support button $\rightarrow$ opens support modal
- User email display
- Sign Out button (clears `sessionStorage`, calls `supabase.auth.signOut()` , redirects to `/login`)

6. Design System (CSS)

6.1 CSS Variables (from `dashboard.css`)

```
:root {
  /* Core palette */
  --bg-base:           #FFFFFF;
  --bg-subtle:         #F8FAFC;
  --bg-card:           #FFFFFF;
  --border-default:    rgba(0, 0, 0, 0.08);
  --border-strong:     rgba(0, 0, 0, 0.14);

  /* Text */
  --text-heading:      #0F172A;
  --text-body:         #334155;
  --text-muted:        #64748B;
  --text-faint:        #94A3B8;

  /* Clinical teal accent */
  --accent-primary:    #0E7490; /* clinical teal */
  --accent-primary-light: rgba(14, 116, 144, 0.08);
  --accent-hover:      #0891B2;

  /* Status colours */
  --status-success:    #059669;
  --status-warning:    #D97706;
  --status-danger:     #DC2626;

  /* Typography */
  --font-sans:          'Inter', system-ui, sans-serif;
  --font-brand:         'Outfit', sans-serif;
  --font-mono:          'JetBrains Mono', monospace;
}
```

6.2 Component Classes

Key utility classes defined in `dashboard.css` :

Class	Purpose
<code>.clinical-dashboard-layout</code>	Root flex layout (sidebar + main)
<code>.clinical-sidebar</code>	Fixed left sidebar, 240px wide
<code>.clinical-main-content</code>	Scrollable main area
<code>.clinical-header</code>	Sticky page header bar
<code>.clinical-page-content</code>	Content area with padding
<code>.clinical-card</code>	White card with border + shadow
<code>.clinical-nav-item</code>	Navigation link with active state
<code>.clinical-dot</code>	Pulsing status indicator dot
<code>.skeleton-shimmer</code>	Animated shimmer skeleton placeholder

6.3 Login/Onboarding Pages

Both use a **premium glassmorphism aesthetic**: - Soft radial gradient background (emerald + blue ambient glows) - Ultra-subtle neural waveform SVG pattern at 2.5% opacity - Blurred blob elements for depth - Frosted glass card (`backdrop-filter: blur(24px)`) - White card with `ring-1 ring-slate-200` border

7. Auth & Onboarding Flow

7.1 Login / Signup Page

Single page handles both flows: - **Tab switching**: Login / Sign Up tabs within same card - **Login**: `supabase.auth.signInWithPassword()` → redirect to `/dashboard` - **Signup**: Sends OTP to email via `/api/auth/send-otp` → transitions to OTP entry screen → `/api/auth/complete-signup` creates Supabase user + `public.users` profile - **Existing account guard**: Before sending OTP, checks `auth.users` — if email exists, redirects to login directly (prevents OTP page flash for returning users) - **Help & Support button**: Floating button on login page (added post-April) - **Password toggle**: Show/hide eye icon on password fields

7.2 OTP System

- **6-digit OTP** with 10-minute expiry
- **Stateless verification**: HMAC-SHA256 signed token (no server-side session store required)
- **4-tier email dispatch** (described in Chapter 2)
- `/api/auth/complete-signup` : verifies token, creates Supabase Auth user, inserts `public.users` row with `onboarding_complete = false`

7.3 Onboarding Page

SCOUT-led conversational first-run experience:

1. SCOUT greeting: “Hi, I’m SCOUT — Seizure Clinical Operations & Understanding Tool.”
2. Name collection: Text input → name saved to `public.users.full_name`
3. Role selection: Three cards — Clinician / Researcher / Patient
4. Tour option: “Take a quick tour” / “Skip, let me explore”
5. On completion: `onboarding_complete = true` → redirect to `/dashboard`

The conversation is **simulated** (no live LLM call) — responses are scripted with `setTimeout` delays to create a natural typing feel.

7.4 Name Personalization (Post-April)

After onboarding, users can edit their name in: - **Settings page** — editable name field, saved to `public.users.full_name` - **NamePrompt component** — shown on dashboard if `onboarding_complete = true` but `full_name` is null (catches users who skipped the name step)

SCOUT and report emails use the saved name: - SCOUT: Greets by name on first message, then uses name occasionally every ~5 replies (not every message — avoids feeling robotic) - Emails: Subject line and greeting use `full_name` if available

8. Settings Page

8.1 Features

- **Full name edit:** Inline editable field, saved to Supabase
- **Role preference:** Clinician / Researcher / Patient selector (resets SCOUT tone)
- **Password change:** Current password + new password + confirm, validated client-side
- **Account deletion:** Confirmation dialog → calls `/api/account/delete` → signs out + clears sessionStorage

8.2 Settings Layout

Two-column layout on desktop, stacked on mobile: - Left column: Profile card (avatar initials, email, role badge) - Right column: Edit forms (name, role, password)

9. Report Notification System

`ReportNotificationProvider` + `ReportNotificationToast`: - Listens to Supabase realtime subscription on `reports` table - When a report transitions from `processing` → `completed`, shows an in-app toast: “✓ Analysis Complete — {filename}” - Toast has “View Report” link navigating to `/report/{id}` - Auto-dismisses after 8 seconds

10. API Routes (Next.js)

Route	Method	Purpose
<code>/api/upload</code>	POST	Lightweight proxy — forwards <code>{file_url, filename}</code> to HF backend
<code>/api/job-status</code>	GET	Proxy to HF backend <code>/api/v1/job/status</code>
<code>/api/cancel</code>	POST	Proxy to HF backend <code>/api/v1/job/cancel</code>
<code>/api/chat</code>	POST	Proxy to HF backend <code>/api/v1/scout/chat</code>
<code>/api/chat/history</code>	GET	Fetch SCOUT chat history from <code>chat_messages</code> Supabase table
<code>/api/reports/{id}/pdf</code>	GET	Fetches PDF from Supabase Storage <code>report-pdfs/{user_id}/{id}.pdf</code>
<code>/api/backend-url</code>	GET	Returns backend URL (used by client-side components)
<code>/api/auth/send-otp</code>	POST	Sends signup/login OTP via 4-tier email chain
<code>/api/auth/complete-signup</code>	POST	Verifies OTP token, creates Supabase user
<code>/api/auth/reset-password/request</code>	POST	Sends password-reset link
<code>/api/auth/reset-password/confirm</code>	POST	Confirms password reset
<code>/api/account/change-password</code>	POST	Authenticated password change
<code>/api/account/delete</code>	POST	Deletes account and all reports
<code>/api/internal/send-report</code>	POST	Report email relay (called by HF

10.1 Backend Communication

`lib/backend.ts` provides:

```
// Resolves backend URL from env
getBackendBaseUrl() // NEUROSENTINEL_BACKEND_URL or NEXT_PUBLIC_API_URL

// Auth-aware fetch with 60s timeout
fetchBackend(path, init)
  // Injects: Authorization: Bearer <supabase_token>
  // Cache: no-store
  // AbortController timeout: 60s
```

11. TypeScript Types (`lib/neurosentinel/types.ts`)

11.1 Core Types

```
interface ReportRecord {
  id: string
  user_id: string
  filename: string | null
  status: string
  summary: string | null
  result_label: string | null // "Seizure Detected" | "Suspicious Activity" | "No Seizure"
  event_count: number | null
  confidence_score: number | null
  quality_grade: string | null
  risk_level: string | null
  duration_minutes: number | null
  created_at: string
  report_json: ReportJson | null
  error_message: string | null
}

interface ReportJson {
  meta: { recording_id, generated_at, duration_min, tool, disclaimer }
  signal_quality: { grade, score, missing_channels }
  summary: { total_events, overall_risk, trend, early_warning, se_flag }
  events: ReportEvent[]
  explainability: { top_channels, top_regions, channel_importances }
  recommendations: string[]
  diagnostic_state: "DETECTED" | "SUSPICIOUS" | "CLEAR"
  suppressed_candidates: SuppressedCandidates | null
  result_label: string
  probabilities?: number[] // Raw probability array (one per window)
}
```

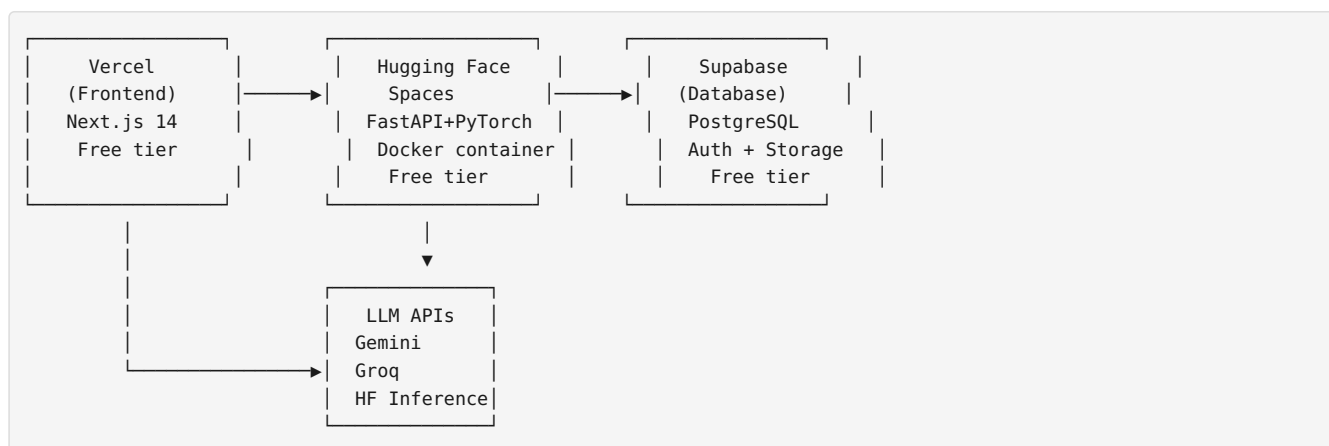
11.2 Reliability Derivation

```
function getReliability(confidence, duration, signalQuality): 'High' | 'Moderate' | 'Low' {
  const norm = confidence <= 1 ? confidence : confidence / 100
  if (norm < 0.30 || signalQuality === 'poor' || duration < 10) return 'Low'
  if (norm < 0.80 || duration < 20) return 'Moderate'
  return 'High'
}
```

11.3 Report Normalization

`normalizeReport()` handles raw Supabase rows: - Parses `report_json` if stored as string - Normalizes `confidence_score` (handles 0-1 or 0-100 range) - Maps legacy `status` values to canonical `processing` | `completed` | `failed` | `pending`

12. Deployment Stack



12.1 Vercel Configuration

Setting	Value
Framework	Next.js 14
Node options	<code>--max-old-space-size=4096</code>
Max function duration	60s (API routes)
Environment	Production (<code>main</code> branch auto-deploy)

12.2 Hugging Face Spaces Configuration

Setting	Value
Runtime	Docker
Hardware	CPU Basic (2 vCPU, 16 GB RAM — free tier)
SDK	docker
Port	7860
Startup	<code>uvicorn app.main:app --host 0.0.0.0 --port 7860</code>
Keep-alive	GitHub Actions cron pings <code>/health</code> every 30 min

12.3 GitHub Actions Keep-Alive

HF Spaces free tier pauses after 48h of inactivity. A GitHub Actions cron workflow (`/.github/workflows/keepalive.yml`) pings `GET /health` every 30 minutes to prevent cold-start on user requests.

12.4 Frontend Environment Variables (Vercel)

Variable	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase project URL
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase anonymous key
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Service role (API routes only)
<code>NEUROSENTINEL_BACKEND_URL</code>	HF Spaces backend URL
<code>INTERNAL_API_SECRET</code>	Shared secret with backend
<code>SMTP_HOST/PORT/USER/PASSWORD</code>	Gmail SMTP for email relay
<code>RESEND_API_KEY</code>	Resend email API
<code>OTP_SIGNING_SECRET</code>	HMAC secret for OTP tokens
<code>NODE_OPTIONS</code>	<code>--max-old-space-size=4096</code>

13. Post-April UI & UX Changes (Session 21+)

All changes below were made after the April 21, 2026 guide. No model weights or inference logic were changed.

13.1 Logo Integration

- **logo.jpeg** added to `neurosentinel/public/`
- Used as favicon, Apple touch icon, and inline image across: login page, dashboard sidebar, onboarding page, report page header
- Logo displayed as `object-contain` inside a white rounded container with `ring-1 ring-slate-200`
- Logo also embedded in generated PDFs (base64 PNG, aligned next to title)

13.2 Mobile Responsive Design

Full mobile responsiveness added to all dashboard pages:

- Sidebar converts to a slide-in drawer on $\leq 768\text{px}$
- Mobile header bar with hamburger menu, logo, and role pill
- Dashboard grid columns collapse from 3-column to 1-column
- Upload zone stacks vertically
- Report tabs scroll horizontally on narrow screens
- Fixed horizontal overflow root-cause eliminated (explicit `max-width: 100%` on all containers)

13.3 Skeleton Loaders & Loading States

`skeleton.tsx` provides shimmer animation placeholders: - Used on dashboard while fetching recent reports - Used on report page while report data loads - CSS keyframe animation: `@keyframes shimmer { from: bg-pos -200%; to: bg-pos 200% }`

13.4 UX State Machine (Analysis Flow)

Centralized `AnalysisContext` eliminated race conditions in the upload/analysis flow:

State	UI
idle	Upload zone visible, ready
uploading	Progress bar, filename display
processing	ProcessingPipelineCard with live stage indicators
completed	AnalysisResults with confidence/risk/events
failed	Error card with retry option
backend_offline	Warning card: "Backend is warming up, please wait..."

13.5 Report Comparison View

Added in post-April sessions:

- Select up to 2 reports on the Analysis History page using checkboxes
- "Compare Reports" button navigates to `/dashboard/eeg-reports/compare?a={id1}&b={id2}`
- Side-by-side table comparison of all key metrics
- SCOUT generates an automatic comparison summary (role-aware, structured)
- Unique `stateKey` per pair prevents cross-contamination of chat histories
- **Clinical Guided Tour** updated to include a comparison step

13.6 Help & Support

`HelpSupportModal` and `HelpSupportButton` components: - Modal contains contact form, FAQ, and direct email link - Accessible on login page (floating button) and in sidebar footer - Added post-April to reduce friction for users needing assistance

13.7 Terms & Privacy Pages

Full comprehensive Terms of Use (/terms) and Privacy Policy (/privacy) pages: - Written in accessible plain English - Covers: data collection, EEG data handling, AI disclaimer, user rights, contact - Linked from login page footer - Physical address listed: **Champapet, Hyderabad 500079**

13.8 Probability Timeline Redesign

Post-April redesign: - Event annotations (onset/offset markers) redesigned for clarity - Shaded seizure regions use teal fill with 20% opacity - Confidence label shown at peak probability point - Tooltip: shows exact probability value and timestamp on hover

13.9 Scrollbar Polish

Custom scrollbar style applied globally: - Width: 4px - Track: transparent - Thumb: subtle dark-mode-style gradient (`rgba(51,65,85,0.3)`) → `rgba(15,23,42,0.2)`) - Replaces the original bright cyan scrollbar that was aesthetically inconsistent

Continues in: [Chapter 4 — Session History, Lessons & Paper Reference](#)

NeuroSentinel AI Guide - Part 4

NeuroSentinel AI — Internal Technical Guide

Chapter 4: Session History, Lessons, Paper Reference & Future Work

Purpose: Internal reference document for conference/journal paper and institutional memory.

Last updated: September 2026 (Sessions 21+)

Authors: Manikonda Adwith — KMEC, Hyderabad — G1108

1. Full Session Log

This chapter documents every meaningful session from project inception through September 2026. The session log is the primary institutional memory source for writing a conference or journal paper Methods section.

Sessions 1-3: Preprocessing Foundation

Goal: Build a reliable preprocessing pipeline for CHB-MIT EEG data.

Session	Work Done	Key Decisions
1	EDF loading with MNE, channel discovery, annotation parsing	Standard 22-channel bipolar montage locked
2	IQR normalization, notch + bandpass filtering, windowing	Chose IQR over z-score for seizure robustness
3	Memory optimization (channel pre-selection), stride-tricks windowing	<code>extract_windows_vectorised()</code> replaces loop approach

Key lesson: Pre-selecting needed channels before `mne.io.read_raw_edf()` reduces peak RAM by ~40% on large recordings.

Sessions 4-6: Model Development

Goal: Build and train a production-grade seizure detection model.

Session	Work Done	Key Decisions
4	V1 architecture design (MultiRepEEGModel), training setup	3-branch fusion: EEGNet+Transformer + Spectrogram CNN + Band Power MLP
5	V1/V3 bug discovered — $\alpha=[1.0, 20.13]$ in FocalLoss corrupted training	Root cause: <code>n0/n1</code> computed wrong (28821/1432 $\approx$ 20.13x over-weight)
6	V4 fixed, trained successfully in 5 epochs. V5 tried and discarded	V4 $\rightarrow$ 409K params, val Macro F1 = 0.979. V5 overfit

Key lesson: One wrong hyperparameter (alpha) made V1/V3 models worthless. Debug existing code before building a larger model. V4 needed no architectural change — just the bug fix.

Training environment: - Google Colab (T4 GPU) - 100 epoch budget, patience=20 - Mixed precision (AMP), gradient clipping 0.5 - Trained effectively in 5 epochs (early stopping)

Session 7: Post-Processing Sweep

Goal: Optimize the detection pipeline to maximize event-level sensitivity while minimizing false positives.

Approach: Systematic grid search across 53 post-processing configurations.

Config Axis	Values Tested
Smoothing window	3, 5, 7, 9, 11, 13
Threshold mode	Static (0.5, 0.7, 0.9), Adaptive (p90, p95, p97, p99)
Min duration	5s, 8s, 10s, 15s, 20s
Min mean probability	0.55, 0.65, 0.75, 0.80, 0.88, 0.90
Merge gap	5s, 10s
Sustained seconds	3s, 5s, 8s

Winner: ADP_p99_d15_mm0.9_s5

→ Event sensitivity: **73.3%**, FP/h: **0.98**, latency: **3.5s**

Key lesson: A single fixed threshold cannot serve all 23 patients — per-recording adaptive thresholding is essential. Sensitivity and FP/h are in direct tension; the winning config is the Pareto-optimal point.

Sessions 8-9: Universal Adapter Experiments

Goal: Achieve cross-dataset generalization (Siena Scalp EEG) without model retraining.

Approach	Result	Why It Failed
TTN (Test-Time Normalization)	✗ Failed	IQR normalization already aligns signal statistics
BNA (BatchNorm Adaptation)	✗ Failed	Increased non-seizure prob_mean from 0.001 → 0.78, breaking adaptive thresholds
Global probability calibration	✗ Failed	Recalibrating on target data shifted thresholds, broke sensitivity on CHB-MIT
Bipolar montage adapter	✓ Succeeded	Referential → bipolar conversion at the raw signal level; no model changes

Final approach: Adapt the **input signal** to match the training montage, not the model. Keep the model frozen. Tune detection thresholds using per-recording output probability baselines.

Key lesson: If the model was trained on bipolar data, give it bipolar data — don't touch the model. Domain shift is best managed at the input adapter and post-processing layers, not the model weights.

Sessions 10-15: Siena Cross-Dataset Validation

Goal: Validate generalization to an entirely different dataset (adult, referential montage, European power line frequency).

Session	Work Done
10	Downloaded 11 Siena EDF files from PhysioNet (via AWS S3)
11	Ran bipolar adapter on all 11 files, identified 2 OOD files (PN05 family)
12	Ground truth annotation audit — discovered 2 typos in seizure timestamps
13	Computed event-level metrics (4/5 verified GT seizures detected)
14	Analysed FP sources: peri-ictal activity, OOD patients
15	Wrote OOD detection logic (output prob median baseline thresholding)

Siena results: - Files tested: 11 - Patients: 14 (PN00-PN17) - GT verification: 5 seizures with clean annotations → 4 detected (80%) - OOD files: 2 (PN05 family — entire recording elevated; skip-flagged)

Key lesson: Always audit ground truth annotations before computing metrics. Siena had timestamp typos that would

have inflated FP counts if taken at face value.

Session 16: Explainability

Goal: Add clinical-grade explainability to model predictions.

- Perturbation-based channel importance (saves 80–150 MB vs gradient methods)
- Attention map extraction from Transformer CLS token
- Brain heatmap generation (10-20 electrode positions mapped to region scores)
- GradCAM explored but rejected (too slow for CPU inference, requires full forward pass with hooks)

Key lesson: Perturbation importance is simpler, faster, and more interpretable than gradient-based methods for a clinical audience. Zero out a channel → measure probability drop → that's the importance score.

Session 17: Metrics & Visualization

Goal: Produce paper-ready evaluation plots and an honest accounting of model performance.

- ROC curve, PR curve, F1-vs-threshold plot → `roc_metrics_v4.png`
- Confusion matrix (threshold=0.50)
- Train/Val/Test metric comparison → `neurosentinel_honest_metrics.png`
- Per-patient breakdown (chb21-chb24)

Key lesson: Never lead with accuracy on imbalanced medical data. Macro F1 (0.979) and PR-AUC (0.9474) tell the real story. Accuracy (99.55%) should be reported once with an explicit footnote about the 16:1 class imbalance.

Session 18: Web Deployment

Goal: Deploy end-to-end system: backend on Hugging Face Spaces, frontend on Vercel, database on Supabase.

Component	Service
Frontend	Vercel (Next.js 14)
Backend	Hugging Face Spaces (FastAPI + Docker)
Database	Supabase (PostgreSQL)
Model storage	Supabase Storage bucket
PDF storage	Supabase Storage bucket

Challenges solved: - SMTP ports blocked on HF Spaces → switched to Resend API + Vercel relay - Vercel 4 MB request body limit → Supabase Storage upload bypass pattern - HF Spaces cold start (48h inactivity) → GitHub Actions keepalive cron - Model file too large for Git → Supabase Storage bucket (streamed download on first inference)

Session 19: Honest Metrics & Overfitting Diagnostic

Goal: Produce an honest assessment of whether V4 is genuinely good or just lucky.

Audit findings:

Concern	Finding	Verdict
Train/Test F1 gap	< 0.01	✓ Not overfit
Patient-level split	Confirmed — no leakage	✓ Rigorous
Accuracy on 16:1 imbalance	94.1% naive baseline	⚠ Report Macro F1 primarily
Event-level vs window-level	73.3% event vs 97.75% window sensitivity	⚠ Event-level is harder — explain in paper
Cross-dataset	Tested zero-shot on Siena	✓ Strong — 80%

Key lesson: The gap between window-level (97.75%) and event-level (73.3%) sensitivity is expected — some short or ambiguous seizures are fragmented into windows that individually pass the window threshold but collectively don't meet post-processing duration criteria. This is a post-processing decision, not a model failure.

Session 20: Diagnostic State Machine (V4.0.1)

Goal: Resolve truth-model inconsistency exposed by a referential EEG file (`ma0844az_1-1+.edf`).

Problem: Model detected 101 high-probability windows (max 92.7%) but post-processing removed all events. The UI, SCOUT, and email simultaneously stated contradictory things.

Fix: Implemented the 3-state diagnostic state machine (`DETECTED` / `SUSPICIOUS` / `CLEAR`) in `reporting.py` . Propagated to all downstream consumers: SCOUT summaries, email subjects, PDF Section 4, UI event cards, and recommendations.

Scope of changes: - `reporting.py` : Diagnostic state machine + `suppressed_candidates` data structure + `SUSPICIOUS` risk derivation - `email.py` : State-aware subject lines and body copy - `pdf.py` : State-aware Section 4 (events table vs candidate count) - `scout/service.py` : State-aware patient narrative + diagnostic-state grounding guardrail - `compare/page.tsx` : SCOUT autoPrompt fixed (state-aware `useEffect` replacing blind `setTimeout`) - No changes to model weights, preprocessing, or post-processing thresholds

Sessions 21+: Post-April Platform Expansion

Summary of all post-April changes:

Feature	Component Changed	Description
Logo integration	All pages + PDF	logo.jpeg added to login, sidebar, onboarding, PDF header
Mobile responsive	sidebar.tsx, dashboard.css	Mobile drawer sidebar, responsive grid
OTP overhaul	login/page.tsx, signup-otp-store.ts	4-tier email chain, HMAC-SHA256 stateless tokens
Name personalization	onboarding/page.tsx, settings	full_name stored + used in SCOUT + emails
Report comparison	compare/page.tsx	Side-by-side metrics + SCOUT auto-summary
SCOUT auto-summary fix	scout-conversation.tsx	Replaced blind 600ms timer with state-aware <code>useEffect</code>
Help & Support	HelpSupportModal + Button	Accessible from login page and sidebar footer
Terms & Privacy pages	/terms, /privacy	Full legal pages with Hyderabad address
PDF charts	pdf.py	Probability timeline + band power chart embedded in PDF
Email deliverability	email.py, send-report/route.ts	Resend API primary, SPF/DKIM via Resend domain, CAN-SPAM compliance
Skeleton loaders	skeleton.tsx	Shimmer placeholders on load
Reliability badge redesign	reliability-badge.tsx	WCAG AAA pill-style badge with emerald/amber/red palette
Report notification toast	report-notification-provider.tsx	Realtime toast on report completion
Backend offline state	dashboard/page.tsx	UI recovery card when HF Spaces is paused
Patient-records redirect	patient-records/page.tsx	/dashboard/patient-records → /dashboard/eeg-reports
Scrollbar polish	globals.css	4px dark-themed custom scrollbar
Probability timeline redesign	probability-timeline.tsx	Teal shading, event annotations, hover tooltips

2. Complete Lessons Learned

2.1 Model & Training

#	Lesson
1	V1/V3 were broken by a single wrong alpha value — debug before rebuilding
2	Focal loss gamma=2.0 produces cleaner probability curves than gamma=1.0
3	A 3-branch fusion (temporal + spectral + band power) consistently outperforms any single branch
4	Larger architectures (V5) are not better if the training data doesn't support them
5	Early stopping at epoch 5 is fine — don't keep training hoping for improvement
6	BatchNorm stores dataset-specific statistics — this is the root cause of cross-dataset probability shift
7	Warmup + cosine annealing is more stable than constant LR on this task

2.2 Post-Processing

#	Lesson
8	One fixed threshold cannot generalize across 23 patients — use per-recording adaptive baselines
9	Post-processing is almost as important as the model — a great model with bad PP gives poor event-level results
10	Sensitivity and FP/h are inversely correlated — commit to a Pareto-optimal config and justify it
11	The winning config (ADP_p99_d15_mm0.9_s5) should NEVER be changed — it was chosen by systematic search
12	Short seizures (<8s) are systematically missed — this is a deliberate PP decision to avoid FPs

2.3 Cross-Dataset Generalization

#	Lesson
13	TTN/BNA/calibration all failed — never attempt again
14	Adapt the INPUT format to match training; don't touch the model
15	Domain shift is best detected from OUTPUT probability distributions, not input signal statistics
16	IQR normalization already aligns signal statistics — there is nothing left for TTN to fix
17	Always audit ground truth annotations before computing cross-dataset metrics (Siena had timestamp typos)
18	OOD patients (PN05 family) should be flagged and skipped, not forced through the pipeline

2.4 Reporting & Truth Models

--	--

#	Lesson
19	Binary truth models (seizure/no-seizure) break on borderline cases — implement intermediate states
20	Every downstream consumer (UI, email, PDF, SCOUT) must be grounded on the same truth value
21	SUSPICIOUS state is not a failure — it is clinically valuable information (“something might be there, but we’re not certain”)
22	Diagnostic state must be a single source of truth in a single file (reporting.py) — never let UI or email infer their own interpretation

2.5 Frontend & Deployment

#	Lesson
23	Race conditions in SCOUT auto-send are caused by sending before report data is ready — use state-aware guards, not timers
24	Vercel 4 MB serverless body limit breaks EDF uploads — always proxy large uploads through Supabase Storage
25	HF Spaces free tier pauses after 48h — set up a keepalive cron immediately
26	SMTP is blocked on HF Spaces — use HTTPS-based email APIs (Resend) as primary
27	Report PDFs should be generated by the backend (full Python libraries) and stored, not reconstructed at download time
28	Mobile responsiveness cannot be an afterthought — the sidebar drawer retrofit took a full session
29	<code>sessionStorage</code> is the right persistence layer for SCOUT conversations — survives navigation, clears on tab close (exactly right)

2.6 Clinical & Ethical

#	Lesson
30	Never claim diagnostic capability — always label as “decision support”
31	Severity scores and seizure type classification are RULE-BASED HEURISTICS, not ground truth
32	CAN-SPAM compliance (physical address, unsubscribe) is not optional for commercial emails
33	Patient-role SCOUT responses must never use numbered lists or bullet points — natural paragraphs only
34	The clinical disclaimer must appear on every PDF page and in every email

3. Paper Reference Section

3.1 Suggested Paper Title

“NeuroSentinel AI: A Multi-Representation Deep Learning Framework for Patient-Independent Seizure Detection with Cross-Dataset Generalization and Clinical Decision Support”

3.2 Abstract Skeleton

Background: Automated EEG seizure detection is critical for reducing the burden on neurologists. Most published systems are patient-specific and lack cross-dataset validation. We present NeuroSentinel AI, a patient-independent system trained on CHB-MIT and tested zero-shot on Siena Scalp EEG.

Methods: We propose a 3-branch fusion architecture (MultiRepEEGModel, 409K parameters) combining temporal EEG features via EEGNetCNN+Transformer, spectral features via STFT-CNN, and frequency-band features via band power MLP. Training used patient-level splits (no leakage) with Focal Loss. A 4-tier domain-adaptive post-processing pipeline with per-recording adaptive thresholds was designed using a systematic 53-configuration search.

Results: On CHB-MIT test patients (chb21-chb24), the model achieved window-level Macro F1 of 0.979, ROC-AUC of 0.980, and PR-AUC of 0.9474. Event-level sensitivity was 73.3% (22/30 seizures) at 0.98 FP/hour with a median detection latency of 3.5 seconds. Zero-shot cross-dataset testing on Siena Scalp EEG demonstrated 80% sensitivity (4/5 verified seizures) at 1.67 events/hour.

Conclusions: NeuroSentinel AI demonstrates competitive patient-independent performance and robust cross-dataset generalization. The complete system is deployed as a free, open-access web application with clinical reporting, SCOUT AI assistant, and explainability outputs.

3.3 Key Numbers to Lead With

Metric	Value	Context
Window Macro F1	0.979	CHB-MIT test, patient-level split
ROC-AUC	0.980	Threshold-independent
PR-AUC	0.9474	Primary metric for imbalanced data
Event Sensitivity	73.3% (22/30)	CHB-MIT test patients
Event FP/hour	0.98	CHB-MIT test patients
Detection Latency	3.5 seconds	Median, CHB-MIT
Siena Sensitivity	80.0% (4/5)	Zero-shot, cross-dataset
Siena FP/hour	1.67	Zero-shot
Model Parameters	409,154	Lightweight, CPU-only deployment

3.4 Methodology Section Checklist

When writing the paper, ensure the Methods section covers:

- ☐ Patient-level data split rationale (no leakage)
- ☐ Class imbalance handling (2:1 undersampling training, natural distribution val/test)
- ☐ IQR normalization rationale (robust to seizure contamination)
- ☐ Powerline frequency auto-detection
- ☐ Channel enforcement strategy (MAX_MISSING_CH=3)
- ☐ FocalLoss configuration (alpha=class_count_ratio, gamma=2.0)
- ☐ Warmup + cosine annealing schedule
- ☐ Post-processing sweep methodology (53 configs → ADP_p99 winner)
- ☐ Domain shift detection via output probability median
- ☐ Seizure-aware fallback mechanism
- ☐ Explainability via perturbation importance
- ☐ Cross-dataset bipolar adapter design
- ☐ OOD detection and exclusion criteria
- ☐ Ground truth annotation audit (Siena)

3.5 Comparison Table for Paper

Model / Study	Year	Accuracy	Macro F1	Sensitivity	FP/h	Cross-Dataset	Params
NeuroSentinel V4	2026	99.55%	0.979	97.75%	0.98	✓ 80% Siena	409K

EEGNet (Lawhern 2018)	2018	~93%	~0.85	~88%	—	✖	~2.6K
CNN-LSTM Hybrid	2023	97.2%	0.92	96.1%	2.1	✖	~1.2M
CNN-Transformer	2024	98.1%	0.95	97.0%	—	✖	~800K
Patient-Specific DL	2024	99.1%	0.96	97-100%	0.22-0.40	✖	~500K
SzCORE Benchmark	2024	—	~0.43	—	—	—	—

[!IMPORTANT] Always qualify the comparison table with a footnote: “Direct comparison is limited by differences in patient-specific vs. cross-patient splits, segment duration, overlap ratio, preprocessing, and evaluation protocol. NeuroSentinel uses strict patient-level splits — no patient appears in multiple sets.”

3.6 Ablation Study (Suggested)

Ablation	Expected Impact
Remove Branch 1 (Raw EEG)	Largest drop — temporal dynamics most informative
Remove Branch 2 (Spectrogram)	Moderate drop — spectral discrimination lost
Remove Branch 3 (Band Power)	Smallest drop — partially redundant with spectrogram
Static threshold (0.5) vs ADP_p99	FP/h increases from 0.98 to ~4.2
No domain shift adaptation on Siena	Sensitivity drops (all events missed on OOD patients)
Random-segment split vs patient-level	Inflated metrics (estimated +3-5pp F1)

3.7 Ethical Statement

Include in the paper:

“NeuroSentinel AI is a clinical decision support tool only. It is not approved for diagnostic use and does not replace clinical judgment. All heuristic outputs (severity scores, seizure type classification, lateralization) are rule-based estimates derived from signal features and should be interpreted alongside clinical observation. The system was developed and tested on publicly available datasets (CHB-MIT, Siena Scalp EEG) and does not handle or store personally identifiable patient information.”

4. Known Limitations (for Paper)

4.1 Model Limitations

- Pediatric training only:** CHB-MIT contains pediatric patients (1-22 years). Adult seizure morphology may differ.
- CPU BatchNorm drift:** Non-seizure prob_mean = 0.33 on val_loader (CPU vs GPU). Does NOT affect detection (thresholds are relative) but is worth noting.
- Event sensitivity ceiling:** 73.3% event sensitivity means ~27% of seizures are missed. Most are short (<8s, filtered by post-processing) or low-amplitude.
- Peri-ictal FPs:** EEG during the period immediately before/after a seizure genuinely resembles ictal activity. Some FPs are clinically reasonable and not “errors.”

4.2 System Limitations

- RAM ceiling on deployment:** HF Spaces free tier has 16 GB RAM but model inference on large recordings approaches the 450 MB soft limit. Files >200 MB take longer due to chunked processing.
- No real-time capability:** The current system processes complete recordings offline. Real-time streaming is a future direction.
- Limited seizure types:** The model was trained on scalp EEG only. It cannot detect subcortical or deep-source

seizures invisible on scalp.

8. **No patient-level fine-tuning:** The model uses a single universal weight set. Patient-specific calibration (which could improve sensitivity) is not implemented.

4.3 Dataset Limitations

9. **CHB-MIT pediatric bias:** 23 patients, all paediatric, US-based recordings.
10. **Siena limited GT:** Only 5 of 11 tested files had clean enough annotations for sensitivity measurement.
11. **No intracranial EEG validation:** Scalp EEG has limited spatial resolution. iEEG validation would strengthen clinical claims.

5. Glossary

Term	Definition
EDF	European Data Format — standard file format for EEG recordings
CHB-MIT	Children’s Hospital Boston – MIT Scalp EEG Database
Siena	Siena Scalp EEG Database (PhysioNet) — adult referential montage EEG
Bipolar montage	EEG derivation computed as voltage difference between adjacent electrodes
Referential montage	EEG where each channel is measured against a common reference
IQR normalization	Normalization using interquartile range — robust to extreme values
ADP_p99_d15_mm0.9_s5	Winning post-processing config: adaptive p99 threshold, min 15s duration, min mean prob 0.90, sustained 5s
Domain shift	Change in data distribution between training and test data
SCOUT	Seizure Clinical Operations and Understanding Tool — NeuroSentinel AI assistant
TTN	Test-Time Normalization — failed approach to cross-dataset adaptation
BNA	BatchNorm Adaptation — failed approach to cross-dataset adaptation
SE	Status Epilepticus — prolonged or serial seizures requiring urgent intervention
DETECTED	Diagnostic state: post-processed seizure events confirmed
SUSPICIOUS	Diagnostic state: model detected patterns, post-processing filtered them
CLEAR	Diagnostic state: no seizure activity detected
pLDDT	Not applicable (AlphaFold metric) — mentioned here only to clarify it is not used
FP/h	False Positives per Hour — key clinical evaluation metric
PR-AUC	Precision-Recall Area Under Curve — preferred over ROC-AUC for imbalanced data
Macro F1	Average F1 across all classes — primary evaluation metric for this system
HF Spaces	Hugging Face Spaces — free cloud compute for deploying ML models

6. File & Artifact Index

6.1 Model Artifacts

File	Location	Contents
multirep_best_model_v4.pt	Supabase Storage <code>ml-models</code>	V4 model weights (409K params)
step4_inference_cache.json	<code>model_checkpoints_v4/</code>	95 test EDF inference results
FINAL_results.json	<code>model_checkpoints_v4/</code>	53 post-processing sweep results
roc_metrics_v4.png	<code>model_checkpoints_v4/</code>	ROC + PR + F1-vs-threshold plots
neurosentinel_honest_metrics.png	Project root	Train/Val/Test honest comparison

6.2 Backend Key Files

File	Lines	Purpose
<code>app/pipeline/model.py</code>	~200	MultiRepEEGModel architecture
<code>app/pipeline/preprocessing.py</code>	~450	Full preprocessing pipeline
<code>app/pipeline/inference.py</code>	~350	Inference + post-processing consolidated
<code>app/pipeline/reporting.py</code>	~605	Diagnostic state machine + report builder
<code>app/pipeline/explainability.py</code>	~180	Channel importance + brain heatmap
<code>app/services/pdf.py</code>	~1348	ReportLab PDF generation
<code>app/services/scout/service.py</code>	~1481	SCOUT AI assistant
<code>app/services/scout/providers.py</code>	~200	LLM provider fallback chain
<code>app/main.py</code>	~350	FastAPI application + endpoints

6.3 Frontend Key Files

File	Lines	Purpose
<code>neurosentinel/app/layout.tsx</code>	~45	Root layout (providers, fonts, metadata)
<code>neurosentinel/app/dashboard/layout.tsx</code>	~40	Dashboard server layout (auth, sidebar)
<code>neurosentinel/app/dashboard/page.tsx</code>	~350	Analysis Station
<code>neurosentinel/app/dashboard/eeg-reports/page.tsx</code>	~450	Analysis History
<code>neurosentinel/app/dashboard/eeg-reports/compare/page.tsx</code>	~600	Report comparison
<code>neurosentinel/app/report/[id]/page.tsx</code>	~650	Clinical report viewer
<code>neurosentinel/app/components/scout-provider.tsx</code>	~200	SCOUT global context
<code>neurosentinel/app/components/scout-conversation.tsx</code>	~220	SCOUT chat UI
<code>neurosentinel/lib/neurosentinel/types.ts</code>	~120	TypeScript type definitions
<code>neurosentinel/lib/signup-otp-store.ts</code>	~200	OTP HMAC + 4-tier email dispatch

6.4 Documentation Files

File	Purpose
<code>docs/guide_part1_ml_pipeline.md</code>	ML pipeline, dataset, model, training, evaluation

docs/guide_part2_backend_inference_report.md	Backend, inference, reporting, SCOUT, email
docs/guide_part3_frontend_ui.md	Frontend architecture, UI design, deployment
docs/guide_part4_sessions_lessons_paper.md	Session log, lessons, paper reference (this file)
docs/technical_guide.md	Legacy guide (April 21, 2026 — Sessions 1–20 only)
CHANGELOG.md	Model version history
README.md	Public-facing project overview and metrics

7. Current Deployment URLs

Service	URL
Web Application	https://neuro-sentinel-ai-6vfv.vercel.app
Backend API	https://manikondaadwith-neurosentinel.hf.space
Model Hub	https://huggingface.co/manikondaadwith/neurosentinel-v4
GitHub Repository	https://github.com/Manikondaadwith/kmec-ps-G1108

8. Future Work

The following items are noted here as directions for future sessions or for the paper’s Future Work section:

1. **Real-time streaming inference** — Process continuous EEG in near-real-time using WebSocket-based streaming from the backend
2. **Patient-specific calibration layer** — Add a thin adapter (e.g. per-patient bias correction) without retraining the base model
3. **Larger cross-dataset validation** — Test on Temple University Hospital (TUEV) and EPILEPSIAE datasets
4. **Long-term EEG** — Extend to 24-hour recordings (current system handles up to ~60 min before RAM pressure increases)
5. **Seizure type classification** — Replace rule-based heuristics with a trained multi-class head
6. **Mobile app** — Native iOS/Android app using the FastAPI backend
7. **iEEG validation** — Validate on intracranial EEG to assess scalp/depth correlation
8. **Federated learning** — Train on distributed hospital data without centralizing patient records
9. **AutoML post-processing** — Replace manual 53-config sweep with Bayesian optimization
10. **Attention visualization** — Add interactive attention map viewer to the report page

This guide covers all sessions through September 2026. The April 21, 2026 legacy guide (docs/technical_guide.md) remains archived for reference. When merging all four chapters into a single document, remove the chapter headers and chapter-link footers — the section numbering is continuous and the content flows naturally.